MLFP mlfp03 — Textbook Chapter

- MLFP Module 3 — Supervised Machine Learning
  - Prerequisites and Notation
  - How to Read This Textbook
  - What You Will Build
  - Table of Contents
- Lesson 3.1 — Feature Engineering, ML Pipeline, and Feature Selection
  - Why This Matters
  - Core Concepts
    - The ML Pipeline
    - Statistics vs Machine Learning
    - Feature Engineering from Domain Knowledge
    - Temporal Features
    - Interaction Terms and Polynomials
    - Leakage: The Silent Killer
    - Feature Selection Taxonomy
  - Mathematical Foundations
    - Mutual Information
    - Chi-Squared Test of Independence
    - Recursive Feature Elimination
  - Kailash Engine: FeatureEngineer and FeatureStore
  - Worked Example: Engineering Features for HDB Resale Prices
  - Try It Yourself
  - Cross-References
  - Deeper Dive: The ExperimentTracker Pattern
  - The Feature Lifecycle
  - Filter Method Details: The Scoring Functions
    - Practical Selection Rule-of-Thumb
  - Reflection Questions
- Lesson 3.2 — Bias-Variance, Regularisation, and Cross-Validation
  - Why This Matters
  - Core Concepts
    - The Target Analogy
    - Mathematical Foundations: The Bias-Variance Decomposition
    - What Drives Bias and Variance?
    - Regularisation
    - Geometric Interpretation: Diamond vs Circle
    - Bayesian Interpretation
    - Cross-Validation
  - Kailash Engine: CrossValidator and PreprocessingPipeline
  - Worked Example: Bias-Variance and Regularisation on Credit Data
    - Nested Cross-Validation Example
  - Try It Yourself
  - Cross-References
  - Deeper Dive: Regularisation Path and the Lasso Solution Path
  - Deeper Dive: Why L2 Has a Closed-Form Solution and L1 Does Not
  - The CV-Standard-Deviation Rule (One-Standard-Error Rule)
  - Reflection Questions
- Lesson 3.3 — The Complete Supervised Model Zoo
  - Why This Matters
  - Core Concepts
    - Support Vector Machines
    - SVM Dual Formulation and the Kernel Trick
    - K-Nearest Neighbours
    - Naive Bayes
    - Decision Trees
    - Random Forests
    - Model Selection Framework
  - Kailash Engine: TrainingPipeline
  - Worked Example: All Five Models on E-Commerce Churn
    - Computing Gini Impurity by Hand
    - OOB Computation Demonstration
  - Try It Yourself
  - Cross-References
  - Deeper Dive: SVM Slack and the Hinge Loss
  - Deeper Dive: KNN Distance Weighting
  - Deeper Dive: Naive Bayes with Laplace Smoothing
  - Deeper Dive: Tree Pruning Mechanics
  - Reflection Questions
- Lesson 3.4 — Gradient Boosting Deep Dive
  - Why This Matters
  - Core Concepts
    - Boosting vs Bagging

        - AdaBoost (Historical Warmup)
        - Gradient Boosting: The General Framework
    - Mathematical Foundations: XGBoost's Second-Order Derivation
        - The Objective
        - Second-Order Taylor Expansion
        - Optimal Leaf Weights
        - The Split Gain Formula
        - Summary of XGBoost Hyperparameters
    - LightGBM: Histograms and GOSS
        - Histogram-Based Split Finding
        - Gradient-Based One-Side Sampling (GOSS)
        - Leaf-Wise Tree Growth
    - CatBoost: Ordered Boosting and Native Categorical Handling
        - Target Leakage in Standard Boosting
        - Ordered Boosting
        - Native Categorical Handling
        - When to Choose Which Library
    - Kailash Engine: TrainingPipeline and AutoMLEngine
    - Worked Example: Credit Scoring with Three Boosters
        - Computing Split Gain by Hand
    - Try It Yourself
    - Cross-References
    - Deeper Dive: Learning Rate and Shrinkage
    - Deeper Dive: Early Stopping
    - Deeper Dive: Understanding Gradients and Hessians by Loss
    - Reflection Questions
- Lesson 3.5 — Model Evaluation, Imbalance, and Calibration
    - Why This Matters
    - Core Concepts
        - The Complete Metrics Taxonomy
        - Why Accuracy Fails on Imbalanced Data
        - Class Imbalance Solutions
        - Focal Loss
        - Probability Calibration
        - Platt Scaling
        - Isotonic Regression
        - Reading a Calibration Plot
    - Kailash Engine: Metrics, Sampling, and Calibration
    - Worked Example: Imbalanced Credit Scoring
    - Try It Yourself
    - Cross-References
    - Deeper Dive: ROC vs PR Curves on Imbalanced Data
    - Deeper Dive: Isotonic Regression by Pool-Adjacent-Violators
    - Deeper Dive: The Cost-Optimal Threshold, Derived
    - Reflection Questions
- Lesson 3.6 — Interpretability and Fairness
    - Why This Matters
    - Core Concepts
        - The Explanation Problem
        - Shapley Values: The Intuition
        - The Four Shapley Axioms
        - The Shapley Value Formula
        - Approximations: KernelSHAP and TreeSHAP
        - SHAP Plots
        - LIME: Local Interpretable Model-Agnostic Explanations
        - Fairness: The Impossibility Theorem
        - Fairness as Engineering
    - Kailash Engine: ModelVisualizer with SHAP
    - Worked Example: SHAP and Fairness Audit on Credit Scoring
    - Try It Yourself
    - Cross-References
    - Deeper Dive: Why Shapley Values Sum to the Prediction
    - Deeper Dive: Computing SHAP for a Single Tree
    - Deeper Dive: Local Accuracy and Consistency
    - Deeper Dive: Fairness Mitigation Techniques
    - Reflection Questions
- Lesson 3.7 — Workflow Orchestration, Model Registry, and Hyperparameter Search
    - Why This Matters
    - Core Concepts
        - The Workflow Abstraction
        - Custom Nodes
        - Logic Nodes: Branching and Conditional Execution
        - Hyperparameter Search
        - Bayesian Optimisation in Depth
        - Kailash HyperparameterSearch
        - Model Registry: Lifecycle Management

        - ModelSignature: The Contract
    - Worked Example: End-to-End Workflow
        - Hyperparameter Search with the Workflow
        - Registry and Promotion
    - Try It Yourself
    - Cross-References
    - Deeper Dive: Bayesian Optimisation with Expected Improvement
    - Deeper Dive: Tree-Structured Parzen Estimator (TPE)
    - Deeper Dive: The Workflow as a Reproducibility Artefact
    - Reflection Questions
- Lesson 3.8 — Production Pipeline: DataFlow, Drift, and Deployment
    - Why This Matters
    - Core Concepts
        - The Full Production Pipeline
        - DataFlow: Persistence for ML Results
        - DriftMonitor: Watching the World Change
        - PSI (Population Stability Index)
        - KS Test (Kolmogorov-Smirnov)
        - Performance Drift
        - DriftSpec
        - Conformal Prediction
        - Model Cards
        - MLOps: Clean Architecture for ML
    - Worked Example: End-to-End Production Pipeline
    - Try It Yourself
    - Cross-References
    - Deeper Dive: Conformal Coverage Guarantee
    - Deeper Dive: Drift Detection Cadence
    - Deeper Dive: Why Three Splits?
    - Deeper Dive: What a Complete Model Card Looks Like
    - Reflection Questions
    - Module 3 Closing
    - Appendix Z: Worked Numerical Examples
        - Z.1: Computing Bias and Variance from Bootstrap Samples
        - Z.2: XGBoost Split Gain with Explicit Numbers
        - Z.3: PSI Computation with Actual Numbers
        - Z.4: Conformal Quantile Calculation
    - Appendix A: End-to-End Singapore Case Study
        - The Problem
        - Lesson 3.1: Features
        - Lesson 3.2: Bias-Variance and Regularisation
        - Lesson 3.3: Model Zoo Comparison
        - Lesson 3.4: Gradient Boosting
        - Lesson 3.5: Imbalance and Calibration
        - Lesson 3.6: Interpretability and Fairness
        - Lesson 3.7: Workflow and Hyperparameter Search
        - Lesson 3.8: Production
    - Appendix B: Formulas Reference
    - Appendix C: Kailash Engine Quick Reference
    - Appendix D: Further Reading

MLFP Module 3 — Supervised Machine Learning

From Feature Engineering to Production Deployment

A textbook for self-study. This module builds the complete supervised ML pipeline — from raw data to a monitored, calibrated, explainable model in production. It assumes you have completed Module 1 (Python foundations and polars) and Module 2 (statistics, probability, and linear regression).

Prerequisites and Notation

This textbook assumes you have completed MLFP Modules 1 and 2. Specifically:

- **From Module 1**: Python basics, polars dataframes, numpy arrays, reading CSV and parquet files, and environment setup with `load_dotenv`.
- **From Module 2**: Descriptive statistics (mean, variance, standard deviation), probability distributions (Gaussian, Bernoulli, Poisson), Bayesian thinking (prior, likelihood, posterior), linear regression including OLS and gradient descent, and hypothesis testing basics.

Notation used throughout:

- `x` is an input vector (features), `x_i` is the i-th observation, `x_j` is the j-th feature.
- `y` is a target scalar, `y_hat` is a model prediction.
- `X` is an input matrix `(n, p)` where `n` is samples and `p` is features.
- `beta` is a coefficient vector for linear models.
- `f(x)` is the true (unknown) function; `y_hat(x)` is the model's approximation.

- `E[.]` denotes expectation, `Var(.)` variance.
- `log` without a base means natural log unless noted; `log_2` is explicit base 2.
- Lowercase `p` is a predicted probability; `p_t` is the probability of the true class.
- Greek letters for parameters: `alpha`, `beta`, `gamma`, `lambda`, `theta`.

Code blocks use Python 3.11+ syntax with polars 0.20+, scikit-learn 1.3+, and kailash-ml 0.4+.

How to Read This Textbook

Each lesson follows the same rhythm:

1. **Why This Matters** — a real ML deployment story that motivates the lesson.
2. **Core Concepts** — intuition first, then definitions, then worked examples.
3. **Mathematical Foundations** — the derivations that matter. Skip on first read if you want the operational story; return for mastery.
4. **Kailash Engine** — which Kailash engine implements the concept and how to call it.
5. **Worked Example** — a full code walkthrough on Singapore credit, HDB, or ICU data.
6. **Try It Yourself** — three exercises of increasing difficulty.
7. **Cross-References** — pointers back to earlier modules and forward to later ones.
8. **Reflection** — four questions to test your understanding.

You will find that Module 3 is long because the pipeline is long. Every lesson is a node in a graph that ends with a deployed model. Do not skip nodes.

What You Will Build

By the end of Module 3, you will have built a complete, production-ready ML system for Singapore credit default prediction. The system will:

- Engineer 80+ domain-aware features from raw credit bureau and transaction data, with automated leakage detection.
- Train, tune, and calibrate a gradient-boosted model achieving ~0.60 average precision on a 12% positive-rate problem.
- Compare five model families (linear, SVM, KNN, Naive Bayes, trees) and three boosting libraries (XGBoost, LightGBM, CatBoost) against one another with a consistent evaluation protocol.
- Explain individual and global predictions using SHAP with the four-axiom Shapley foundation.
- Measure fairness across protected attributes using disparate impact, equalized odds, and calibration parity, and document trade-offs via the impossibility theorem.
- Orchestrate the entire pipeline as a reproducible Kailash workflow with Bayesian hyperparameter search and model registry lifecycle.
- Persist evaluation results and predictions to a database via DataFlow.
- Monitor deployed models for input and prediction drift using PSI and KS tests.
- Generate a 9-section Mitchell et al. model card.
- Produce conformally calibrated prediction sets with a 90% coverage guarantee.

This is the complete production ML stack. You will not just read about it — you will build it.

Table of Contents

- Lesson 3.1 — Feature Engineering, ML Pipeline, and Feature Selection
- Lesson 3.2 — Bias-Variance, Regularisation, and Cross-Validation
- Lesson 3.3 — The Complete Supervised Model Zoo
- Lesson 3.4 — Gradient Boosting Deep Dive
- Lesson 3.5 — Model Evaluation, Imbalance, and Calibration
- Lesson 3.6 — Interpretability and Fairness
- Lesson 3.7 — Workflow Orchestration, Model Registry, and Hyperparameter Search
- Lesson 3.8 — Production Pipeline — DataFlow, Drift, and Deployment

---

Lesson 3.1 — Feature Engineering, ML Pipeline, and Feature Selection

Why This Matters

In 2019, a team at a Singapore bank deployed a credit default model that used 212 features. The model achieved an AUC of 0.91 on the validation set. It went live on a Monday. By Friday, the risk team noticed something strange: the model was predicting default risk almost perfectly for loans that were already in collections, but performed no better than a coin flip on fresh applications.

The investigation revealed a single feature: `days_since_last_payment_reminder`. It was computed from the collections system, which only contained entries for customers who had already defaulted. The feature was a near-perfect predictor of the target, but it was only available *after* the outcome was known. The model had learned a tautology: “customers who have received collection reminders are likely to default.” That feature was a **leak**.

The bank rebuilt the model with 87 features, none of which referenced post-application events. Validation AUC dropped to 0.78. Live AUC held at 0.76 — a ten-point drop from the leaked version, but a model that actually worked. The team saved approximately S$14 million in fraud losses over the next twelve months. Not by choosing a better algorithm. By choosing better features.

This lesson is about feature engineering — the activity that more than any other distinguishes an ML project that works in production from one that looks impressive on a slide deck. You will learn:

- Why domain knowledge beats model complexity.
- How to engineer features from raw data without creating leaks.
- How to select which features to keep using filter, wrapper, and embedded methods.
- How to use Kailash's `FeatureEngineer` and `FeatureStore` to make this reproducible.

Remember the deck 4A ordering: **Data > Models > Hyperparameter Tuning**. Time spent on features returns more than time spent on hyperparameters by roughly an order of magnitude.

Core Concepts

The ML Pipeline

Every supervised ML project has the same eight stages. Memorise them — they will appear in every exercise in this module.

1. **Data ingestion** — pulling raw data from databases, APIs, or files.
2. **Preprocessing** — cleaning missing values, encoding types, handling outliers.
3. **Feature engineering** — deriving new columns that carry domain-meaningful signal.
4. **Feature selection** — reducing the feature set to those that actually help.
5. **Model selection** — choosing an algorithm family (trees? linear? boosted?).
6. **Training and evaluation** — fitting on training data and measuring on held-out data.
7. **Hyperparameter tuning** — searching for the best configuration of the chosen algorithm.
8. **Deployment and monitoring** — serving predictions and detecting drift.

The bulk of a successful ML engineer's time is spent on stages 2 through 4. Stages 5 through 7 are increasingly automated. Stage 8 is where the most recent wave of tooling (MLOps) lives. This lesson covers stages 3 and 4 in depth.

Statistics vs Machine Learning

There is a deep philosophical difference between statistics and ML that matters for how you approach feature engineering.

- **Statistics** asks: *why did this happen?* The goal is to explain the past. The model is a lens on the data-generating process. Features are chosen because they have interpretable coefficients.
- **Machine learning** asks: *what will happen next?* The goal is to predict the future. The model is a function that maps inputs to outputs. Features are chosen because they improve out-of-sample accuracy.

A statistician studying HDB prices might include the floor area because it has a causal relationship with value per square metre. An ML engineer might include floor area, but also include the *ratio* of floor area to the median floor area in the same block, because that ratio is a stronger predictor even though it has no clean causal interpretation. Both approaches are valid. They answer different questions.

Module 2 taught you statistics. Module 3 teaches you ML. Hold both ways of thinking in your head.

Feature Engineering from Domain Knowledge

A feature is any derived column you compute from the raw data. Good features have three properties:

1. **Domain relevance** — a domain expert would nod when you describe the feature.
2. **Predictive power** — the feature meaningfully reduces uncertainty about the target.
3. **Availability at prediction time** — the feature can be computed when you need to score a new observation, not just in hindsight.

Let us look at a concrete example. Suppose you are predicting HDB resale prices. The raw data includes:

- `address` — a string like "456 Ang Mo Kio Ave 10".
- `floor_area_sqm` — integer square metres.
- `lease_commence_date` — year the lease started.
- `resale_date` — when the sale happened.

From these four columns, a domain-aware feature engineer might derive:

- `lat`, `lon` — geocoded from the address via OneMap API. Location is the single most important HDB feature.
- `distance_to_mrt_m` — Euclidean distance from (lat, lon) to the nearest MRT station.
- `distance_to_cbd_km` — distance to Raffles Place.
- `remaining_lease_years` — `(lease_commence_date + 99) - resale_date.year`. HDB leases are 99-year leasehold; remaining lease is a strong price driver.
- `price_per_sqm_block_median_12m` — the median resale price per square metre in the same block over the past 12 months. This is a leading indicator of local market momentum.
- `month`, `quarter`, `year` — extracted from `resale_date` to capture temporal trends and seasonality.

Notice what each feature encodes: location, accessibility, asset age, local market context, and time. None of these columns existed in the raw data. All of them require you to know something about Singapore housing.

Temporal Features

Time is special. When your target depends on when an event happens, you almost always want to decompose time into multiple features.

- **Lag features**: the target's value at a previous time step. For forecasting, `sales_t-1`, `sales_t-7`, `sales_t-30` are standard.
- **Rolling features**: statistics over a window. `rolling_mean_7d`, `rolling_std_14d`, `rolling_max_30d`.

- **Calendar features**: `day_of_week`, `day_of_month`, `month`, `quarter`, `is_weekend`, `is_holiday`.
- **Elapsed time**: `days_since_signup`, `days_until_payment_due`, `age_in_years`.
- **Cyclical encoding**: `sin(2*pi*day_of_year/365.25)` and `cos(2*pi*day_of_year/365.25)` to preserve the fact that December 31 and January 1 are adjacent.

A word of warning: **lag and rolling features are the most common source of leakage**. If you compute a rolling mean over a window that includes the prediction day, you are using the future to predict the present. Always ensure your window ends strictly before the prediction timestamp.

Interaction Terms and Polynomials

Sometimes two features carry more information together than apart. The interaction between `floor_area_sqm` and `distance_to_cbd_km` is a stronger price driver than either feature alone, because space is worth more when it is central.

You can encode interactions directly:

```
df = df.with_columns(
    (pl.col("floor_area_sqm") * pl.col("distance_to_cbd_km")).alias("area_x_distance"),
    (pl.col("floor_area_sqm") ** 2).alias("area_squared"),
)
```

Polynomial features (`x`, `x^2`, `x^3`) let a linear model capture curvature. Tree-based models discover interactions automatically, so you need interaction terms less often for trees. For linear models, they are essential.

Be careful not to generate thousands of polynomial features on a small dataset. A degree-3 polynomial on 50 features creates `C(50+3, 3) = 22,100` new columns. That is the recipe for overfitting.

Leakage: The Silent Killer

A **feature leak** is any feature that would not be available at the moment you needed to make a prediction. The bank story that opened this lesson is a textbook example: `days_since_last_payment_reminder` was not known at application time.

Common leakage patterns:

- **Target leakage**: the feature is computed using the target itself. If you one-hot encode `customer_segment` where `segment` was assigned based on whether the customer churned, you have target leakage.
- **Temporal leakage**: the feature is computed using data from after the prediction time. Rolling means that include the prediction day. Normalisation statistics computed over train + test combined.
- **Multi-tenancy leakage**: the feature depends on other customers' outcomes. `mean_default_rate_by_zip` computed including the target customer.
- **Identifier leakage**: the feature is a proxy for the row's identity. `customer_id` in a one-hot encoding. An index column that happens to correlate with the target because the data was sorted.

The cure is disciplined, point-in-time correct feature construction. For every feature, ask: *could I compute this feature at prediction time using only data that existed before the prediction timestamp?* If the answer is no, drop the feature.

Feature Selection Taxonomy

You engineered 400 features. Which ones should you actually use? Feature selection methods fall into three families.

Filter methods rank features by a univariate score and keep the top N. They are fast and model-agnostic.

- **Pearson correlation** with the target (regression).
- **Mutual information**: `I(X; Y) = H(Y) - H(Y|X)`. Captures non-linear relationships.
- **Chi-squared**: for categorical features and categorical targets.
- **Variance threshold**: drop features with near-zero variance.

Filter methods are fast but miss interactions. A feature that is useless alone but powerful in combination with another will be discarded.

Wrapper methods evaluate subsets of features by actually training a model. They are slow but capture interactions.

- **Forward selection**: start empty, add the feature that improves validation score most, repeat.
- **Backward elimination**: start with all features, remove the one whose removal hurts least, repeat.
- **Recursive Feature Elimination (RFE)**: train a model, drop the feature with the smallest coefficient or importance, repeat.

Wrapper methods are the gold standard for interaction-sensitive selection but cost `O(k * n_features)` model trainings.

Embedded methods let the model itself choose features as part of training.

- **L1 regularisation (Lasso)**: drives coefficients of unhelpful features to exactly zero.
- **Tree importance**: decision trees, random forests, and gradient boosting machines all produce feature importance scores.
- **Elastic Net**: L1 + L2 combination, picks grouped features.

Embedded methods are usually the best trade-off: faster than wrappers, smarter than filters.

In practice, a good workflow is: variance threshold → mutual information filter to 2x the target feature count → L1 or tree-based embedded method for the final cut.

Mathematical Foundations

Mutual Information

Given two random variables `X` and `Y`, the mutual information between them is:

```
I(X; Y) = Sum_{x,y} p(x, y) * log(p(x, y) / (p(x) * p(y)))
```

Equivalently, `I(X; Y) = H(Y) - H(Y | X)`, where `H` is entropy. Mutual information measures how much knowing `X` reduces your uncertainty about `Y`.

- If `X` and `Y` are independent, `p(x, y) = p(x) * p(y)`, so the log term is zero, so `I = 0`.
- If `X` perfectly determines `Y`, `H(Y | X) = 0`, so `I = H(Y)`, the maximum.

Mutual information is symmetric, non-negative, and captures non-linear dependencies. Unlike Pearson correlation, it does not assume linearity. It is the most principled univariate filter for feature selection.

Chi-Squared Test of Independence

For a categorical feature `X` with `c` categories and a categorical target `Y` with `k` classes, construct a contingency table of observed counts `O_{ij}`. Under the null hypothesis that `X` and `Y` are independent, the expected count in cell `(i, j)` is:

```
E_{ij} = (row_i_total * col_j_total) / grand_total
```

The chi-squared statistic is:

```
chi^2 = Sum_{i,j} (O_{ij} - E_{ij})^2 / E_{ij}
```

It has `(c - 1)(k - 1)` degrees of freedom. A high chi-squared rejects independence, meaning `X` is informative about `Y`. Scikit-learn's `SelectKBest(chi2)` uses this statistic directly.

Recursive Feature Elimination

Given a model `M` that produces per-feature importance scores `w_i`, RFE proceeds:

1. Train `M` on all `p` features.
2. Rank features by `|w_i|`.
3. Remove the feature with the smallest `|w_i|`.
4. Retrain `M` on the remaining `p - 1` features.
5. Repeat until only `k` features remain.

RFE is a greedy approximation to the optimal subset problem, which is NP-hard. It often produces good subsets because important features tend to have stable rankings across re-training.

Kailash Engine: FeatureEngineer and FeatureStore

Kailash provides `FeatureEngineer` to automate the generation of derived features and `FeatureStore` to version and retrieve them. Both are part of the `kailash_ml` package.

```python
from kailash_ml import FeatureEngineer, FeatureStore
from kailash_ml.types import FeatureSchema, FeatureField

# Declare the contract
schema = FeatureSchema(
    name="hdb_resale_v1",
    fields=[
        FeatureField("lat", dtype="float64", nullable=False),
        FeatureField("lon", dtype="float64", nullable=False),
        FeatureField("distance_to_mrt_m", dtype="float64", nullable=False),
        FeatureField("remaining_lease_years", dtype="int64", nullable=False),
        FeatureField("price_per_sqm_block_median_12m", dtype="float64", nullable=True),
    ],
)

engineer = FeatureEngineer(schema=schema)
features = engineer.transform(raw_df)
engineer.validate(features)
```

The `FeatureStore` takes this further: it registers features with lineage information, versions them, and lets multiple downstream models pull the same feature set. This is how you avoid the bug where two data scientists derive `remaining_lease_years` slightly differently and get different numbers.

Worked Example: Engineering Features for HDB Resale Prices

The worked example for this lesson uses the HDB resale transactions dataset. We will engineer features, detect leaks, and apply three feature selection methods.

```python
import polars as pl
from kailash_ml import FeatureEngineer, DataExplorer
from kailash_ml.types import FeatureSchema, FeatureField
from shared import MLFPDataLoader
from shared.kailash_helpers import setup_environment
```

```
setup_environment()

loader = MLFPDataLoader()
hdb = loader.load("mlfp01", "hdbprices.csv")
print(hdb.shape)
print(hdb.columns)
```

Start by exploring the data with `DataExplorer`:

```
explorer = DataExplorer()
report = explorer.profile(hdb)
print(report.missing_summary)
print(report.type_summary)
```

The profiler confirms: no nulls in `floor_area_sqm` or `resale_price`, 1.2% nulls in `flat_model`, and `resale_date` is currently a string.

Now engineer temporal features:

```
hdb = hdb.with_columns(
    pl.col("resale_date").str.to_date("%Y-%m"),
).with_columns(
    pl.col("resale_date").dt.year().alias("year"),
    pl.col("resale_date").dt.month().alias("month"),
    pl.col("resale_date").dt.quarter().alias("quarter"),
    ((pl.col("lease_commence_date") + 99) - pl.col("resale_date").dt.year())
        .alias("remaining_lease_years"),
)
```

Notice we use `dt.year()` on the date rather than string manipulation — polars handles dates natively.

Next, compute block-level rolling statistics. This is the step where leakage is most likely, so we are careful:

```
hdb = hdb.sort("resale_date")

block_rolling = (
    hdb.group_by_dynamic(
        "resale_date",
        every="1mo",
        period="12mo",
        by="block",
        closed="left",  # Critical: exclude the current day
    )
    .agg(
        (pl.col("resale_price") / pl.col("floor_area_sqm"))
            .median()
            .alias("price_per_sqm_block_median_12m")
    )
)

hdb = hdb.join(block_rolling, on=["block", "resale_date"], how="left")
```

The `closed="left"` parameter is the point-in-time correctness knob. It says: the window `[t - 12 months, t)` excludes `t` itself. Without this, the current transaction's price contaminates the rolling median, creating a leak.

Engineer interaction features:

```
hdb = hdb.with_columns(
    (pl.col("floor_area_sqm") * pl.col("remaining_lease_years"))
        .alias("area_x_lease"),
    (pl.col("floor_area_sqm") / pl.col("price_per_sqm_block_median_12m"))
        .alias("area_vs_block"),
)
```

Now apply three feature selection methods and compare them.

Filter: mutual information.

```
from sklearn.feature_selection import mutual_info_regression

numeric_cols = [c for c in hdb.columns if hdb[c].dtype.is_numeric() and c != "resale_price"]
X = hdb.select(numeric_cols).drop_nulls()
y = hdb.select("resale_price").to_series()[:X.height]

mi_scores = mutual_info_regression(X.to_numpy(), y.to_numpy(), random_state=42)
mi_ranked = sorted(zip(numeric_cols, mi_scores), key=lambda p: -p[1])
top10_mi = [name for name, _ in mi_ranked[:10]]
print("Filter top 10:", top10_mi)
```

Wrapper: Recursive Feature Elimination.

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression

rfe = RFE(LinearRegression(), n_features_to_select=10)
```

```
rfe.fit(X.to_numpy(), y.to_numpy())
top10_rfe = [numeric_cols[i] for i in range(len(numeric_cols)) if rfe.support_[i]]
print("Wrapper top 10:", top10_rfe)
```

Embedded: Lasso.

```
from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X.to_numpy())
lasso = LassoCV(cv=5, random_state=42).fit(X_scaled, y.to_numpy())
nonzero = [numeric_cols[i] for i in range(len(numeric_cols)) if abs(lasso.coef_[i]) > 1e-6]
print("Embedded (non-zero):", nonzero)
```

Compare the three sets. You will typically see a core of features that all three methods agree on — those are your keepers. Features that only one method selects are candidates for deeper investigation.

Try It Yourself

Exercise A (easy). Using the HDB dataset, engineer the following features and verify each has a non-trivial correlation with `resale_price`:

1. `price_per_sqm` — the resale price divided by the floor area.
2. `lease_age_years` — the age of the lease at resale time.
3. `log_price` — the natural log of the resale price.

Which transformation produces the most symmetric distribution? Why?

Exercise B (medium). Write a function `detect_temporal_leak(df, target, timestamp_col, feature)` that returns `True` if the feature value at row `i` can be computed using only rows where the timestamp is strictly before `df[i, timestamp_col]`. Test it on the `price_per_sqm_block_median_12m` feature from the worked example.

Exercise C (hard). Apply forward selection manually: starting from the empty set, at each step add the feature that most improves 5-fold CV R-squared on a linear regression. Stop when no feature improves the score. Compare the final set to the Lasso-selected set.

Cross-References

- **Module 1, Lesson 1.6** introduced polars date operations.
- **Module 2, Lesson 2.4** covered linear regression, the model used for Lasso here.
- **Module 2, Lesson 2.3** covered Bayesian priors, which will return in Lesson 3.2 as the Bayesian interpretation of L2.
- **Forward link:** Lesson 3.2 uses the features engineered here to demonstrate regularisation.
- **Forward link:** Lesson 3.7 shows how to encode this whole feature engineering step as a Kailash `WorkflowBuilder` node.

Deeper Dive: The ExperimentTracker Pattern

Every feature engineering decision should leave a trail. The MLFP03 ex_1 solution introduces `ExperimentTracker` to log every experiment — which features were generated, which were selected, which were dropped and why. Here is the pattern:

```
from kailash_ml.engines.experiment_tracker import ExperimentTracker

tracker = ExperimentTracker(experiment_name="mlfp03_credit")
with tracker.start_run(run_name="feature_engineering_v1") as run:
    run.log_param("num_raw_features", len(raw_cols))
    run.log_param("geocoding_api", "onemap")
    run.log_param("temporal_window_days", 365)

    features = engineer_all_features(raw_df)
    run.log_param("num_generated_features", len(features.columns))

    selected = select_with_lasso(features, target)
    run.log_param("num_selected_features", len(selected))
    run.log_metric("lasso_alpha", alpha)
    run.log_metric("train_r2", train_r2)

    run.log_artifact("selected_features.json", selected_feature_list)
```

Three months later, when your manager asks “which features did we use in version 1?”, you open the experiment tracker and see exactly the answer. No guessing.

The Feature Lifecycle

Features have their own lifecycle, separate from models:

1. **Proposed**: a domain expert or engineer suggests a feature. Document the rationale.
2. **Generated**: compute the feature on historical data. Verify it exists, is not all nulls, and has reasonable distribution.
3. **Validated**: ensure no leakage, no correlation-to-target that looks suspicious, and acceptable missing rate.
4. **Stored**: register in the `FeatureStore` with a version and schema.
5. **Consumed**: used by one or more downstream models.

6. **Deprecated**: when a better feature replaces it, mark as deprecated but keep computing for models still using it.
7. **Retired**: no models use it; stop computing.

The `FeatureStore` is what makes this lifecycle manageable. Without it, features become implicit — buried in training code, rediscovered by every new team member, silently diverging when multiple people re-implement the same concept.

Filter Method Details: The Scoring Functions

Each filter method scores features individually. Here are the formulas and when to prefer each:

Pearson correlation — linear relationships, continuous-continuous:

```
rho(X, Y) = Cov(X, Y) / (sigma_X * sigma_Y)
```

Range `[-1, 1]`. Rank by `|rho|`. Misses non-linear relationships and won't find a quadratic signal.

Spearman rank correlation — monotonic relationships:

```
rho_S = Pearson correlation of ranks of X and Y
```

Captures any monotonic relationship, not just linear. Preferred when features have outliers or skewed distributions.

Mutual information — any relationship:

```
I(X; Y) = integral p(x, y) log(p(x, y) / (p(x) p(y))) dx dy
```

Catches non-linear and non-monotonic relationships. The best general-purpose filter.

ANOVA F-statistic — continuous feature, categorical target:

```
F = (between-group variance) / (within-group variance)
```

Good for classification when features are continuous. `SelectKBest(f_classif)` in scikit-learn.

Chi-squared — categorical feature, categorical target:

```
chi^2 = Sum (O_ij - E_ij)^2 / E_ij
```

Requires non-negative features and is strictly for categorical-categorical.

Practical Selection Rule-of-Thumb

- Too many features (thousands)? Start with variance threshold to drop constants, then mutual information to rank, then take top 100–200.
- Moderate features (50–500)? Jump straight to L1 (Lasso) for linear problems or tree-based importance for non-linear.
- Few features (< 50)? Probably do not need selection at all. Train all features and let regularisation manage complexity.

Reflection Questions

1. The bank that opened this lesson saw a ten-point AUC drop when they removed the leaked feature. Was that a loss or a gain for the business?
2. When would you prefer a filter method over an embedded method?
3. Why does `closed="left"` in the group_by_dynamic prevent leakage? What goes wrong if you use `closed="right"`?
4. A junior colleague argues that `customer_id` should be included as a feature because one-hot encoding it gives 99% validation accuracy. Explain, in two sentences, what is wrong with this argument.
5. List three features you could engineer for HDB prices that would require external data. For each, describe the data source.
6. You compute mutual information on a 10-million-row dataset and it takes 3 hours. Your colleague suggests subsampling to 100,000 rows. Is the MI estimate on the subsample trustworthy?

Lesson 3.2 — Bias-Variance, Regularisation, and Cross-Validation

Why This Matters

In 2017, a Singapore fintech trained a revenue forecasting model using 180 features and a 12-degree polynomial expansion. On the training set, the R-squared was 0.998. On the validation set, it was 0.41. The CTO asked why.

The model had memorised the training data. Every wiggle, every outlier, every noise blip had been fit perfectly. When new data arrived, the model had no idea what to do. The technical term for this is **overfitting**. The underlying concept is called the **bias-variance tradeoff**, and it is the single most important theoretical idea in supervised learning.

Understanding bias-variance is not optional. Every practical choice in ML — regularisation strength, tree depth, number of features, size of training set, type of cross-validation — is a choice about bias and variance. If you know where you are on the bias-variance curve, you know what to do next.

This lesson covers:

- The formal bias-variance decomposition for squared loss.

- Regularisation as a bias-variance knob — L1, L2, and elastic net.
- The Bayesian interpretation: regularisation as a prior.
- Cross-validation strategies for different data structures.
- Nested cross-validation for unbiased model selection.

Core Concepts

The Target Analogy

Picture a dartboard. You throw darts at the bullseye. Two things can go wrong:

- Your darts cluster tightly, but all off to one side. You are **biased**: something about your aim is systematically wrong.
- Your darts are centred on the bullseye on average, but scattered widely. You have high **variance**: your throw is unreliable.

These two failure modes are different. A biased archer needs to adjust aim. A high-variance archer needs to steady their hand. Fixing bias is a different operation from fixing variance.

In ML, bias and variance correspond to two ways a model can fail:

- **High bias = underfitting.** The model is too simple to capture the structure of the data. A constant function (predict the mean for everyone) is the maximum-bias, zero-variance model.
- **High variance = overfitting.** The model is so flexible that it fits noise in the training set. A lookup table that memorises every training point is the minimum-bias, maximum-variance model.

The bias-variance tradeoff says: *you cannot minimise both at the same time*. Reducing bias usually increases variance, and vice versa. The art of ML is finding the sweet spot.

Mathematical Foundations: The Bias-Variance Decomposition

Suppose the true data-generating process is `y = f(x) + epsilon`, where `epsilon` is zero-mean noise with variance `sigma^2`. The noise is irreducible — no model can fit it. You train a model `y_hat(x)` on a training set `D`. The model depends on `D`, so think of `y_hat` as a random variable over training sets.

At a fixed test point `x_0`, the expected squared error, taken over the randomness of the training set and the noise, is:

```
E[(y - y_hat(x_0))^2]
```

where the expectation is over both the noise `epsilon` and the training set `D`. Let us decompose this.

Step 1. Expand the square:

```
E[(y - y_hat)^2] = E[((y - E[y_hat]) - (y_hat - E[y_hat]))^2]
```

where I have added and subtracted `E[y_hat]`.

Step 2. Expand again:

```
= E[(y - E[y_hat])^2] - 2 E[(y - E[y_hat])(y_hat - E[y_hat])] + E[(y_hat - E[y_hat])^2]
```

Step 3. The cross term vanishes. Because `y = f(x_0) + epsilon` where `epsilon` is independent of the training set:

```
E[(y - E[y_hat])(y_hat - E[y_hat])]
  = E[(f(x_0) + epsilon - E[y_hat])(y_hat - E[y_hat])]
  = (f(x_0) - E[y_hat]) * E[y_hat - E[y_hat]] + E[epsilon] * E[y_hat - E[y_hat]]
  = 0 + 0
  = 0
```

The first zero is because `E[y_hat - E[y_hat]] = 0` by definition of expectation. The second is because `E[epsilon] = 0`.

Step 4. We are left with:

```
E[(y - y_hat)^2] = E[(y - E[y_hat])^2] + E[(y_hat - E[y_hat])^2]
```

Step 5. Expand the first term:

```
E[(y - E[y_hat])^2] = E[(f(x_0) + epsilon - E[y_hat])^2]
                    = E[(f(x_0) - E[y_hat])^2] + 2 E[epsilon (f(x_0) - E[y_hat])] + E[epsilon^2]
                    = (f(x_0) - E[y_hat])^2 + 0 + sigma^2
```

The cross term vanishes because `epsilon` is independent of the model and has mean zero. The last term is `Var(epsilon) = sigma^2` since `E[epsilon] = 0`.

Step 6. Identify the pieces:

- `(f(x_0) - E[y_hat])^2` is `Bias^2(y_hat)`: the squared difference between the true function and the average model.
- `E[(y_hat - E[y_hat])^2]` is `Var(y_hat)`: the variance of the model predictions across training sets.
- `sigma^2` is the irreducible noise floor.

Final decomposition:

```
E[(y - y_hat(x_0))^2] = Bias^2(y_hat(x_0)) + Var(y_hat(x_0)) + sigma^2
```

Read this formula carefully. It says: the expected test error of your model is the sum of three terms — how systematically wrong it is on average (bias squared), how much it jitters with different training sets (variance), and the fundamental unpredictability of the world (noise).

You cannot reduce `sigma^2`. You trade bias against variance. That is the whole game.

What Drives Bias and Variance?

- **Model complexity.** Simple models (linear regression with 3 features) have high bias and low variance. Complex models (a 12-degree polynomial, a deep tree, a large neural net) have low bias and high variance. This is the primary knob.
- **Training set size.** Variance decreases with more data. Bias is unaffected by data quantity for a fixed model family. If you are overfitting, "get more data" almost always helps.
- **Feature quality.** Better features reduce bias without increasing variance — this is why feature engineering is such a good investment.
- **Regularisation.** Adding a penalty to the loss function increases bias and decreases variance. Its strength is the tuning knob.

Regularisation

Take a linear regression loss function:

```
L(beta) = Sum_i (y_i - x_i^T beta)^2
```

Add a penalty term:

```
L_ridge(beta) = Sum_i (y_i - x_i^T beta)^2 + lambda * Sum_j beta_j^2
L_lasso(beta) = Sum_i (y_i - x_i^T beta)^2 + lambda * Sum_j |beta_j|
L_enet(beta)  = Sum_i (y_i - x_i^T beta)^2 + lambda * (alpha * Sum_j |beta_j| + (1 - alpha) * Sum_j beta_j^2)
```

- `lambda` controls the overall strength of regularisation. Larger `lambda` = smaller coefficients = more bias, less variance.
- `alpha` in elastic net blends L1 and L2. `alpha = 1` is pure Lasso, `alpha = 0` is pure Ridge.

The two penalties look similar but behave very differently.

Geometric Interpretation: Diamond vs Circle

Consider minimising the unregularised loss subject to a constraint on the coefficients.

- **L2 constraint**: `Sum_j beta_j^2 <= t`. The feasible region is a ball (a circle in 2D).
- **L1 constraint**: `Sum_j |beta_j| <= t`. The feasible region is an octahedron (a diamond in 2D) with corners on the coordinate axes.

Now picture the level curves of the unregularised loss — concentric ellipses around the unconstrained optimum. As you shrink `t`, the loss ellipses eventually hit the constraint region. The optimum of the constrained problem is the point where they first touch.

- With an L2 ball, the ellipses can touch at any angle. The optimum is a generic point with all coefficients non-zero.
- With an L1 diamond, the ellipses are very likely to touch at a corner — which is on a coordinate axis — meaning one or more coefficients are exactly zero.

This is why Lasso produces sparse solutions: the diamond has corners, the circle does not. Sparsity is not magic; it is geometry.

Bayesian Interpretation

Regularisation has a deep Bayesian interpretation. Recall from Module 2.3 that the posterior is proportional to the likelihood times the prior:

```
p(beta | D) proportional_to p(D | beta) * p(beta)
```

Taking logs:

```
log p(beta | D) = log p(D | beta) + log p(beta) + constant
```

Maximum a posteriori (MAP) estimation maximises this. If we put a **Gaussian prior** on each coefficient — `beta_j ~ N(0, tau^2)` — then:

```
log p(beta) = -Sum_j beta_j^2 / (2 tau^2) + constant
```

Negating to turn maximisation into minimisation, the MAP loss is:

```
-log p(D | beta) + Sum_j beta_j^2 / (2 tau^2)
```

This is exactly Ridge regression with `lambda = 1 / (2 tau^2)`. **L2 regularisation is a Gaussian prior on the coefficients.**

Similarly, if we put a **Laplace prior** — `beta_j ~ Laplace(0, b)` — then:

```
log p(beta) = -Sum_j |beta_j| / b + constant
```

The MAP loss becomes Lasso regression. **L1 regularisation is a Laplace prior on the coefficients.**

Why does this matter? Because it tells you what regularisation is doing philosophically: it encodes a belief that the true coefficients are small before you see the data. Ridge says "I believe coefficients are Gaussian-small". Lasso says "I believe most coefficients are exactly zero, and those that are non-zero can be any size". These are different beliefs about the world, and the choice should be driven by your actual prior.

For a tall, thin dataset (few features, many observations), regularisation barely matters — the data dominates the prior. For a wide dataset (many features, few observations), the prior is load-bearing, and the choice of prior is a genuine design decision.

Cross-Validation

How do you know if your model is generalising? You hold out data. The standard pattern is k-fold cross-validation:

1. Shuffle the data (carefully — see below for exceptions).
2. Split into `k` equal folds.
3. For `i = 1, 2, ..., k`: train on the `k-1` folds excluding fold `i`, evaluate on fold `i`.
4. Average the `k` evaluation scores.

This gives you `k` estimates of test error from `k` different train-test splits. The average is more reliable than a single split.

Why k-fold is unbiased. Assuming the data are i.i.d., each fold is a random sample from the true distribution. The model trained on `k-1` folds is trained on `(k-1)/k` of the data. For `k = 5`, that is 80%. The error estimate is slightly pessimistic because you are evaluating a model trained on less data than you would use at deployment, but as `k -> n` (leave-one-out), this bias vanishes.

Bessel's correction analogue. When estimating the variance of CV scores, you divide by `k - 1` rather than `k`. This is the sample-variance correction you met in Module 2.2 and it applies here for the same reason: you used the sample mean to compute deviations, losing one degree of freedom.

Stratified k-fold. For classification with imbalanced classes, pure random splits can produce folds with very different class proportions. Stratified k-fold preserves the class ratio in every fold. Always prefer stratified for classification.

Time-series split. If your data has a temporal dimension, random shuffling breaks causality — you end up training on future data to predict the past. Use **walk-forward validation** instead: train on `[t_0, t_i]`, evaluate on `[t_i + 1, t_{i+1}]`, roll forward.

GroupKFold. Sometimes observations belong to groups that should stay together. Same patient measured on multiple visits, same company with multiple quarterly filings, same HDB block. Leaking a single group across train and test will wildly overestimate performance. GroupKFold ensures all observations from one group go into the same fold.

Nested cross-validation. When you tune hyperparameters, you need two loops of CV:

- The inner loop searches hyperparameters on the training portion of the outer split.
- The outer loop evaluates the tuned model on held-out data.

Without nesting, the hyperparameter search sees the test data through the tuning process, leading to optimistic bias. Nested CV is expensive (`k_outer * k_inner * n_configs` model fits) but gives honest generalisation estimates.

Kailash Engine: CrossValidator and PreprocessingPipeline

Kailash's `PreprocessingPipeline` wraps the common train-time operations (scaling, encoding, imputation) so you can compose them with any model. Crucially, the pipeline is fit on the training fold only, then applied to the validation fold — avoiding the classic leak where you fit a scaler on all the data.

```
from kailash_ml import PreprocessingPipeline
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.linear_model import Ridge

pipe = PreprocessingPipeline(
    numeric_features=["floor_area_sqm", "remaining_lease_years"],
    categorical_features=["flat_type", "town"],
    scaler="standard",
    encoder="onehot",
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(
    Ridge(alpha=1.0),
    pipe.fit_transform(X),
    y,
    cv=cv,
    scoring="r2",
)
print(f"CV R^2: {scores.mean():.3f} +/- {scores.std():.3f}")
```

Worked Example: Bias-Variance and Regularisation on Credit Data

The MLFP03 ex_2 solution demonstrates bias-variance on Singapore credit scoring data with a continuous target (`credit_utilisation`). Let us walk through a simplified version.

```
import numpy as np
import polars as pl
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge, Lasso, ElasticNet, LinearRegression
from sklearn.model_selection import train_test_split, KFold
from sklearn.metrics import mean_squared_error
from shared import MLFPDataLoader

loader = MLFPDataLoader()
credit = loader.load("mlfp02", "credit_scoring.parquet")
```

```
numeric = ["age", "income", "debt_ratio", "num_credit_lines", "employment_years"]
X = credit.select(numeric).drop_nulls().to_numpy()
y = credit.select("credit_utilisation").to_series().to_numpy()[:len(X)]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Now run a complexity sweep: polynomial features from degree 1 to degree 8, each fit with plain linear regression, and measure both training and test error.

```
train_errors, test_errors = [], []
for degree in range(1, 9):
    pipe = Pipeline([
        ("poly", PolynomialFeatures(degree)),
        ("scaler", StandardScaler()),
        ("lr", LinearRegression()),
    ])
    pipe.fit(X_train, y_train)
    train_errors.append(mean_squared_error(y_train, pipe.predict(X_train)))
    test_errors.append(mean_squared_error(y_test, pipe.predict(X_test)))

for d, (tr, te) in enumerate(zip(train_errors, test_errors), start=1):
    print(f"degree {d}: train MSE = {tr:.4f}, test MSE = {te:.4f}")
```

You will typically see training error decrease monotonically with degree, while test error forms a U-shape: high at both extremes (underfit and overfit) with a minimum in the middle. The minimum of the test curve is the bias-variance sweet spot.

Now compare regularisation strategies at a fixed high degree:

```
degree = 6
poly = PolynomialFeatures(degree)
scaler = StandardScaler()

X_tr_poly = scaler.fit_transform(poly.fit_transform(X_train))
X_te_poly = scaler.transform(poly.transform(X_test))

models = {
    "No reg (LR)": LinearRegression(),
    "Ridge (L2)": Ridge(alpha=1.0),
    "Lasso (L1)": Lasso(alpha=0.01, max_iter=10000),
    "ElasticNet": ElasticNet(alpha=0.01, l1_ratio=0.5, max_iter=10000),
}

for name, model in models.items():
    model.fit(X_tr_poly, y_train)
    mse = mean_squared_error(y_test, model.predict(X_te_poly))
    n_nonzero = np.sum(np.abs(model.coef_) > 1e-6) if hasattr(model, "coef_") else "-"
    print(f"{name:15s}  test MSE = {mse:.4f}  non-zero coefs = {n_nonzero}")
```

Observations to expect:

- Plain LR will have the highest test MSE (overfit).
- Ridge will have lower MSE and all coefficients non-zero but shrunk.
- Lasso will have comparable MSE and many coefficients exactly zero.
- Elastic Net will land somewhere between Ridge and Lasso.

Nested Cross-Validation Example

```
from sklearn.model_selection import GridSearchCV

outer = KFold(n_splits=5, shuffle=True, random_state=42)
inner = KFold(n_splits=3, shuffle=True, random_state=42)

param_grid = {"alpha": [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]}

outer_scores = []
for train_idx, test_idx in outer.split(X):
    X_out_tr, X_out_te = X[train_idx], X[test_idx]
    y_out_tr, y_out_te = y[train_idx], y[test_idx]

    search = GridSearchCV(Ridge(), param_grid, cv=inner, scoring="neg_mean_squared_error")
    search.fit(X_out_tr, y_out_tr)

    best = search.best_estimator_
    score = mean_squared_error(y_out_te, best.predict(X_out_te))
    outer_scores.append(score)
    print(f"  fold alpha = {search.best_params_['alpha']}, test MSE = {score:.4f}")

print(f"Nested CV MSE: {np.mean(outer_scores):.4f} +/- {np.std(outer_scores):.4f}")
```

The nested score is your honest generalisation estimate. It is typically 5–15% worse than a flat CV score because the flat score is tainted by the tuning procedure.

Try It Yourself

Exercise A (easy). Using the bias-variance code above, plot `train_mse` and `test_mse` versus polynomial degree. Mark the sweet spot. Explain in one sentence what each region of the plot represents.

Exercise B (medium). For the Lasso model at degree 6, print the names of the features with non-zero coefficients, sorted by absolute coefficient value. Lasso should have selected a sparse subset — which features did it keep?

Exercise C (hard). Implement a manual bias-variance estimator: bootstrap 100 training sets of size 500 from the credit dataset, train a model on each, and compute the prediction at a fixed test point. The variance of the 100 predictions is the variance component. The squared difference between the average prediction and the true value is the bias squared. Do this for Ridge with `alpha = 0.01` and `alpha = 100`. Which has higher bias? Which has higher variance?

Cross-References

- **Module 2, Lesson 2.3** Bayesian priors and posteriors — directly connects to the Bayesian interpretation of regularisation.
- **Module 2, Lesson 2.4** Linear regression — the base model we are regularising.
- **Module 2, Lesson 2.2** Bessel's correction — appears again in CV variance estimates.
- **Forward link:** Lesson 3.3 and 3.4 will rely on CV for model comparison.
- **Forward link:** Lesson 3.7 will use Bayesian optimisation to search the hyperparameter space.

Deeper Dive: Regularisation Path and the Lasso Solution Path

Lasso and Ridge have a beautiful property: as you sweep `lambda` from infinity down to zero, the coefficients trace a piecewise-linear path for Lasso (piecewise-linear because the L1 penalty is piecewise-linear) and a smooth curve for Ridge. This path reveals which features enter the model first.

```
from sklearn.linear_model import lars_path
import matplotlib.pyplot as plt

alphas, _, coefs = lars_path(X_train, y_train, method="lasso")
for i, feat in enumerate(feature_names):
    plt.plot(-np.log10(alphas + 1e-6), coefs[i], label=feat)
plt.xlabel("-log10(alpha)")
plt.ylabel("Coefficient")
plt.legend()
```

Features that enter early (at high `lambda`) are the most important — Lasso picks them first. Features that enter last have marginal contributions. This is a visual feature-importance ranking that comes for free with Lasso.

Deeper Dive: Why L2 Has a Closed-Form Solution and L1 Does Not

The Ridge loss is smooth everywhere. Setting its gradient to zero gives a closed-form solution:

```
beta_ridge = (X^T X + lambda * I)^(-1) X^T y
```

This always exists because `X^T X + lambda * I` is positive definite for `lambda > 0`. No iterative optimisation needed.

The Lasso loss has kinks at `beta_j = 0` because `|beta_j|` is not differentiable there. No closed form exists. Instead, Lasso requires iterative algorithms:

- **Coordinate descent**: update one coefficient at a time, holding others fixed. Each update is a soft-threshold operation. Scikit-learn's default.
- **LARS (Least Angle Regression)**: traces the entire regularisation path efficiently by following the kinks.
- **ISTA/FISTA**: proximal gradient methods.

The absence of a closed form makes Lasso slightly slower, but the benefit — sparse solutions — is often worth it.

The CV-Standard-Deviation Rule (One-Standard-Error Rule)

Suppose 5-fold CV gives MSE of (3.2, 3.4, 3.1, 3.5, 3.3) for `alpha = 1` and (3.0, 3.1, 3.2, 3.3, 3.1) for `alpha = 10`. The mean for `alpha = 10` is lower, so you pick it. But look at the standard deviations — they overlap. Is the difference statistically meaningful?

Breiman's **one-standard-error rule** says: pick the simplest model whose CV score is within one standard error of the best. "Simplest" usually means the most regularised. The rule's logic: all models within one SE are statistically indistinguishable, so pick the one with the best generalisation properties, which is typically the most regularised.

```
scores = cross_val_score(Ridge(alpha=alpha), X, y, cv=5, scoring="neg_mean_squared_error")
mean = -scores.mean()
se = scores.std() / np.sqrt(len(scores))
print(f"alpha={alpha}: MSE = {mean:.3f} +/- {se:.3f}")
```

Pick the largest `alpha` whose `mean` is within `min_mean + se_at_min` of the best. This protects you from overfitting your hyperparameter search.

Reflection Questions

1. If you have 10 observations and 100 features, should you use Ridge or Lasso? Why?
2. A colleague reports 5-fold CV accuracy of 0.91 for their tuned XGBoost. You report 0.87 from nested 5x3 CV on the same data. Whose number is the honest generalisation estimate? Why are they different?
3. You are predicting quarterly earnings for 100 public companies. You have 40 quarters of history. Which CV strategy do you use? Why?
4. Write the Bayesian interpretation of Elastic Net. What is the corresponding prior?

5. Derive the closed-form Ridge solution by differentiating the Ridge loss with respect to `beta` and setting the gradient to zero.
6. A regularised model with `lambda = 0` is just unregularised. A model with `lambda -> infinity` predicts the mean. Intuitively, where is "enough regularisation"? How do you find it?

Lesson 3.3 — The Complete Supervised Model Zoo

Why This Matters

In 2018, a Singapore e-commerce company was building a customer churn model. The ML team's first attempt used logistic regression. It worked reasonably well — 0.74 AUC. The head of data science insisted they try "something more modern". They tried a deep neural network. It reached 0.76 AUC after two weeks of tuning. They tried XGBoost. It reached 0.83 AUC in an afternoon. They tried a random forest. It reached 0.82 AUC in twenty minutes.

What the team had re-learned is one of the most stable empirical facts in ML: **tree-based ensembles beat neural networks on tabular data**, almost always, almost every time. This is not a prediction about the future — it is an observation from thousands of Kaggle competitions, production deployments, and benchmark studies.

But the story has another layer. When the team shipped the random forest, the legal team asked: "Can you explain why the model rejected this specific customer?" The random forest could not give a simple answer. They ended up adding a decision tree as a fallback model for the "explainability path" — slightly worse accuracy, but every prediction came with a rule.

This lesson is about the complete supervised model zoo: Support Vector Machines, K-Nearest Neighbours, Naive Bayes, Decision Trees, and Random Forests. Each has a mathematical soul, a best-case scenario, and a failure mode. You need all five in your vocabulary.

Core Concepts

Support Vector Machines

Imagine two clouds of points in 2D — one labelled positive, one negative. A linear classifier is a line that separates them. If the points are linearly separable, there are infinitely many such lines. Which one should you choose?

SVM picks the line that has the **widest margin** — the largest possible gap between the line and the nearest points from each class. The points that sit on the margin edges are called **support vectors**; they are the only points that matter for the classifier. Move any other point slightly, and the decision boundary does not change.

Why the widest margin? Because wider margins generalise better. A narrow margin means the boundary is sensitive to perturbations; a wide margin gives the boundary room to be wrong without flipping predictions.

The hard-margin SVM. For linearly separable data, the optimisation problem is:

```
minimise   (1/2) ||w||^2
subject to y_i (w^T x_i + b) >= 1  for all i
```

The decision function is `f(x) = sign(w^T x + b)`. The margin width is `2 / ||w||`, so minimising `||w||^2` maximises the margin.

The soft-margin SVM. Real data is usually not linearly separable. We allow some points to violate the margin by introducing slack variables `xi_i >= 0`:

```
minimise   (1/2) ||w||^2 + C * Sum_i xi_i
subject to y_i (w^T x_i + b) >= 1 - xi_i,  xi_i >= 0
```

The parameter `C` controls the trade-off: small `C` tolerates many margin violations (smoother boundary, higher bias), large `C` punishes violations (tighter boundary, higher variance). `C` is a regularisation knob.

SVM Dual Formulation and the Kernel Trick

The primal problem is convex quadratic in `w` and `b`. Its Lagrangian dual is:

```
maximise   Sum_i alpha_i - (1/2) Sum_{i,j} alpha_i alpha_j y_i y_j (x_i^T x_j)
subject to Sum_i alpha_i y_i = 0,  0 <= alpha_i <= C
```

The solution gives `w = Sum_i alpha_i y_i x_i`, and only the points with `alpha_i > 0` contribute — the support vectors. The prediction for a new point `x` is:

```
f(x) = sign(Sum_i alpha_i y_i (x_i^T x) + b)
```

Notice the inner products `x_i^T x`. Everything the dual cares about is this inner product. Now comes the trick: **replace `x_i^T x_j` with any symmetric positive semi-definite function `K(x_i, x_j)`**. This implicitly maps the data into a higher-dimensional space and does the linear SVM there, without ever computing the coordinates in that space.

Common kernels:

- **Linear**: `K(x, y) = x^T y`. Same as no kernel.
- **Polynomial**: `K(x, y) = (gamma x^T y + r)^d`. Captures polynomial interactions up to degree `d`.
- **RBF (Gaussian)**: `K(x, y) = exp(-gamma ||x - y||^2)`. The default for non-linear SVM. `gamma` controls the width of the Gaussian.
- **Sigmoid**: `K(x, y) = tanh(gamma x^T y + r)`. Rarely used in practice.

When to use SVM. Small-to-medium datasets (up to ~50k observations), high-dimensional features, clear margin of separation. Text classification was a classic SVM win before transformers dominated. SVMs do not scale well to millions of rows (training is `O(n^2)` to `O(n^3)`) and their hyperparameters (`C`, `gamma`) require careful tuning.

K-Nearest Neighbours

KNN is the simplest supervised learning algorithm imaginable. To predict the label of a new point `x`:

1. Compute the distance from `x` to every point in the training set.
2. Find the `k` nearest neighbours.
3. For classification, return the majority class among the `k` neighbours. For regression, return their average.

There is no "training" in the usual sense — you just memorise the training set. Everything happens at prediction time.

Distance metrics. The choice of distance matters.

- **Euclidean**: `d(x, y) = sqrt(Sum_j (x_j - y_j)^2)`. The default.
- **Manhattan**: `d(x, y) = Sum_j |x_j - y_j|`. Prefers axis-aligned differences.
- **Cosine**: `d(x, y) = 1 - (x^T y) / (||x|| ||y||)`. Useful when magnitudes do not matter (text, embeddings).

Choosing k. Small `k` = low bias, high variance. Large `k` = high bias, low variance. The standard procedure: cross-validate over `k in {1, 3, 5, 7, 11, 15, 21, 31}` and pick the best. Use odd values for binary classification to avoid ties.

The curse of dimensionality. KNN has a serious flaw in high dimensions. In a `d`-dimensional unit hypercube, the volume of a small ball around any point is `O(r^d)`, which vanishes as `d` grows. To capture any fraction of the data, the ball must grow so large that it is no longer "local". Equivalently: **all pairs of points become roughly equidistant** as `d -> infinity`.

More formally: let `x_1, x_2, ..., x_n` be independent uniform draws from `[0, 1]^d`. Let `r_min` be the distance from the origin to the nearest point and `r_max` the distance to the farthest. Then:

```
(r_max - r_min) / r_min -> 0  as d -> infinity
```

The "nearest" neighbour is no longer meaningfully nearer than the farthest. KNN breaks. In practice, KNN is useless beyond about 30–50 features unless you first reduce dimensionality.

When to use KNN. Small datasets, low dimensionality, smooth decision boundaries. Image similarity search in feature space. A quick baseline before anything fancy.

Naive Bayes

Naive Bayes applies Bayes' theorem with a strong assumption:

```
P(y | x_1, x_2, ..., x_d) proportional_to P(y) * P(x_1, ..., x_d | y)
```

The naive assumption is that the features are conditionally independent given the class:

```
P(x_1, ..., x_d | y) = Prod_j P(x_j | y)
```

This is almost never true in practice. Features are correlated: income and education, age and experience. Naive Bayes ignores all that and multiplies independently.

Amazingly, it often works well anyway. The classifier does not need the probabilities to be calibrated — it just needs to rank classes correctly. Getting the ranking right is a much weaker requirement than getting the probabilities right, and the independence assumption often preserves rankings even when it corrupts calibration.

Variants.

- **GaussianNB**: assume each feature is Gaussian within each class. Works for continuous features.
- **MultinomialNB**: assume features are counts (e.g., word counts in a document). Classic for text.
- **BernoulliNB**: assume features are binary (word present / absent).

When to use Naive Bayes. Text classification (spam filters, sentiment analysis). A fast baseline to compare everything else against. Low-data regime where you cannot afford to overfit.

Decision Trees

A decision tree is a flowchart. Each internal node asks a question ("`income > 50000`?"), each leaf predicts a label. To classify a new point, walk from the root to a leaf, following the branches whose conditions are satisfied.

How trees are built. Start with all training points in the root. For each feature and each candidate threshold, measure how much the split reduces impurity. Choose the feature-threshold pair that reduces impurity most. Split the node into two children. Recurse on each child. Stop when a stopping criterion is met (max depth, minimum samples per leaf, purity threshold).

Impurity measures for classification.

Gini impurity:

```
G = 1 - Sum_k p_k^2
```

where `p_k` is the fraction of samples in class `k` at a node. A pure node (all one class) has `G = 0`; a maximally mixed two-class node (50/50) has `G = 0.5`.

Entropy:

```
H = -Sum_k p_k log_2 p_k
```

A pure node has `H = 0`; a 50/50 split has `H = 1`.

Information gain is the entropy drop from a split:

```
IG = H(parent) - Sum_j (|child_j| / |parent|) * H(child_j)
```

Gini and entropy usually produce similar trees. Gini is cheaper to compute (no log), so it is the default in scikit-learn. Entropy has a cleaner information-theoretic interpretation. Prefer Gini unless you have a reason to use entropy.

For regression, the impurity measure is variance: a split is good if it reduces the within-node variance.

Pre-pruning (stopping early). Limit the tree before it overfits.

- `max_depth`: maximum number of levels.
- `min_samples_split`: don't split a node with fewer samples.
- `min_samples_leaf`: require at least this many samples in each leaf.
- `max_leaf_nodes`: cap the total number of leaves.

Post-pruning. Grow the tree to full depth, then prune back subtrees that do not improve validation accuracy. Cost-complexity pruning (Breiman et al.) is the standard algorithm: `R_alpha(T) = R(T) + alpha |T|`, balancing error against tree size.

When to use decision trees. Interpretability is the main reason. A single tree's predictions can be explained as an if-then-else chain. They handle mixed numeric and categorical data without preprocessing. They are weak alone but form the atoms of random forests and gradient boosting.

Random Forests

A single decision tree has high variance — it changes a lot when you perturb the training data. Ensemble many trees and you get a model that is far more stable. Random Forests are bagged decision trees with two tweaks:

1. **Bootstrap aggregation (bagging)**: train each tree on a random bootstrap sample of the data (sample with replacement, same size as the original). Because bootstraps differ, the trees differ.
2. **Feature subsampling**: at each split, consider only a random subset of features. This decorrelates the trees, because different trees are forced to use different features.

At prediction time, the forest votes (for classification) or averages (for regression). The forest's variance is much lower than a single tree's, while bias is roughly the same. That is why random forests generalise better.

Out-of-bag estimation. Because each bootstrap sample is drawn with replacement from `n` observations, each observation has probability `(1 - 1/n)^n` of being excluded. As `n -> infinity`, this probability approaches `e^{-1} ≈ 0.368`. So about 36.8% of the observations are out-of-bag (OOB) for each tree. You can use the OOB samples as a free validation set: for each observation, average the predictions of only the trees that did not see it. This gives you a generalisation estimate without holding out data.

Feature importance. Random forests report feature importance by measuring how much each feature contributes to impurity reduction across all trees. This is a free side effect of training; no extra computation needed.

When to use Random Forests. Default tabular model when you want a strong baseline with minimal tuning. Robust to feature scaling, handles missing values gracefully, and rarely catastrophically wrong. Beats most specialised models on small-to-medium datasets.

Model Selection Framework

When should you use each model family? Here is a rough decision guide.

- **Linear/logistic with regularisation**: when you want interpretability, speed, and a baseline. Start here.
- **SVM**: medium-sized data, high-dimensional features, when a clear margin exists. Text, bioinformatics.
- **KNN**: small data, low dimensions, smooth boundaries. Rarely a production choice.
- **Naive Bayes**: text, fast baseline, very small data. Good calibration is not a goal.
- **Decision Trees**: interpretability is the primary requirement.
- **Random Forests**: strong tabular default, minimal tuning, robust.
- **Gradient Boosting (Lesson 3.4)**: maximal tabular performance, accepts tuning cost.

Kailash Engine: TrainingPipeline

Kailash's `TrainingPipeline` wraps the scikit-learn ecosystem with consistent APIs for polars dataframes. You declare the model, the preprocessor, the CV strategy, and the metrics; it runs the loop.

```
from kailash_ml import TrainingPipeline
from sklearn.ensemble import RandomForestClassifier

pipeline = TrainingPipeline(
    model=RandomForestClassifier(n_estimators=200, oob_score=True, random_state=42),
    preprocessor=PreprocessingPipeline(
        numeric_features=num_cols,
```

```
        categorical_features=cat_cols,
    ),
    cv_strategy="stratified_kfold",
    cv_folds=5,
    scoring=["accuracy", "f1", "roc_auc"],
)

result = pipeline.fit(X_train, y_train)
print(f"OOB score: {result.model.oob_score_:.3f}")
print(f"CV AUC: {result.cv_scores['roc_auc'].mean():.3f}")
```

Worked Example: All Five Models on E-Commerce Churn

Following the MLFP03 ex_3 solution, we train SVM, KNN, Naive Bayes, Decision Tree, and Random Forest on the same churn dataset and compare.

```
import time
import polars as pl
import numpy as np
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold, cross_val_score
from shared import MLFPDataLoader

loader = MLFPDataLoader()
df = loader.load("mlfp03", "ecommerce_customers.parquet")

numeric = ["age", "total_purchases", "avg_order_value", "days_since_last_order",
           "sessions_per_month", "support_tickets"]
X = df.select(numeric).drop_nulls().to_numpy()
y = df.select("churned").to_series().to_numpy()[:len(X)]

X = StandardScaler().fit_transform(X)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

models = {
    "SVM (RBF)":    SVC(kernel="rbf", C=1.0, gamma="scale", random_state=42),
    "KNN (k=7)":    KNeighborsClassifier(n_neighbors=7),
    "GaussianNB":   GaussianNB(),
    "DecTree":      DecisionTreeClassifier(max_depth=8, random_state=42),
    "RandForest":   RandomForestClassifier(n_estimators=200, oob_score=True, random_state=42),
}

results = []
for name, model in models.items():
    start = time.perf_counter()
    auc_scores = cross_val_score(model, X, y, cv=cv, scoring="roc_auc")
    elapsed = time.perf_counter() - start
    results.append({
        "model": name,
        "auc_mean": auc_scores.mean(),
        "auc_std": auc_scores.std(),
        "time_sec": elapsed,
    })
    print(f"{name:12s}  AUC = {auc_scores.mean():.3f} +/- {auc_scores.std():.3f}   ({elapsed:.1f}s)")
```

You will typically see Random Forest winning on AUC, Decision Tree close behind but higher variance across folds, SVM competitive if you tuned `C` and `gamma`, KNN lagging if the feature count is high, and Naive Bayes at the bottom but training in milliseconds.

Computing Gini Impurity by Hand

Suppose a node contains 100 samples: 70 retained, 30 churned. The Gini impurity is:

```
G = 1 - (70/100)^2 - (30/100)^2
  = 1 - 0.49 - 0.09
  = 0.42
```

Now split on `sessions_per_month > 15`:

- Left child (fewer sessions): 40 samples, 10 retained, 30 churned. `G_L = 1 - 0.0625 - 0.5625 = 0.375`.
- Right child (more sessions): 60 samples, 60 retained, 0 churned. `G_R = 1 - 1 - 0 = 0`.

Weighted child impurity:

```
G_children = (40/100) * 0.375 + (60/100) * 0 = 0.15
```

Gini drop (analogous to information gain):

```
Delta G = 0.42 - 0.15 = 0.27
```

A good split. The child on the right is pure — any sample reaching it is predicted retained with certainty. The child on the left still has a mixed population (10 retained, 30 churned) and will be split again by a child node.

OOB Computation Demonstration

```
rf = RandomForestClassifier(n_estimators=500, oob_score=True, random_state=42)
rf.fit(X, y)
print(f"OOB accuracy: {rf.oob_score_:.3f}")

# Compare with 5-fold CV
cv_acc = cross_val_score(rf, X, y, cv=5, scoring="accuracy").mean()
print(f"5-fold CV accuracy: {cv_acc:.3f}")
```

The two numbers are usually within a percentage point of each other. OOB gives you a free CV estimate; for small datasets, that is valuable because CV splits the data further.

Try It Yourself

Exercise A (easy). Train KNN with `k = 1, 3, 5, 11, 21, 51` on the churn data. Plot AUC vs `k`. What shape does the curve take? Where is the optimal `k`?

Exercise B (medium). Compute the curse of dimensionality empirically. Sample 1000 points uniformly from `[0, 1]^d` for `d in {2, 5, 10, 50, 200}`. For each `d`, compute the ratio `(r_max - r_min) / r_min` where `r_min` and `r_max` are the distances from the origin to the nearest and farthest points. Plot the ratio against `d`. How fast does it shrink?

Exercise C (hard). Write `manual_rf_predict(trees, X_new)` that takes a list of trained decision trees and a new observation, and returns the majority-vote prediction by calling each tree's `predict`. Then compute a confidence score as the fraction of trees that voted for the majority class. Test your function against scikit-learn's `RandomForestClassifier.predict_proba`.

Cross-References

- **Module 2, Lesson 2.3** Bayesian thinking connects directly to Naive Bayes.
- **Lesson 3.2** Cross-validation and bias-variance — used to compare the five models here.
- **Forward link:** Lesson 3.4 takes the Random Forest intuition and replaces bagging with boosting.
- **Forward link:** Lesson 3.6 uses SHAP to explain the Random Forest from this lesson.

Deeper Dive: SVM Slack and the Hinge Loss

The soft-margin SVM can be re-expressed as minimising a loss function: **hinge loss** plus L2 regularisation.

Starting from the constrained form with slack variables `xi_i`:

```
minimise (1/2) ||w||^2 + C * Sum_i xi_i
subject to y_i (w^T x_i + b) >= 1 - xi_i, xi_i >= 0
```

At optimum, `xi_i = max(0, 1 - y_i (w^T x_i + b))`. Substituting back:

```
minimise (1/2) ||w||^2 + C * Sum_i max(0, 1 - y_i (w^T x_i + b))
```

Define the hinge loss `L_hinge(y, f(x)) = max(0, 1 - y f(x))`. The SVM objective becomes:

```
(1/2) ||w||^2 + C * Sum_i L_hinge(y_i, f(x_i))
```

This is just L2-regularised hinge loss. The hinge loss is zero when the prediction is correct and confidently beyond the margin (`y f(x) >= 1`), positive otherwise. This view unifies SVM with other linear classifiers — logistic regression is L2-regularised log loss, SVM is L2-regularised hinge loss. Only the loss function differs.

Deeper Dive: KNN Distance Weighting

Standard KNN gives each of the `k` neighbours an equal vote. Distance-weighted KNN gives closer neighbours more weight:

```
prediction(x) = Sum_{i in N_k(x)} w_i y_i / Sum_{i in N_k(x)} w_i
```

with `w_i = 1 / d(x, x_i)^2` or `w_i = exp(-d(x, x_i)^2 / h^2)` (Gaussian kernel).

Distance weighting lets you use larger `k` without blurring the local estimate, because far-away neighbours barely contribute. Scikit-learn: `KNeighborsClassifier(weights="distance")`.

Deeper Dive: Naive Bayes with Laplace Smoothing

MultinomialNB counts word occurrences per class. If a word never appears in class `C` during training but appears in a test document, the likelihood `P(word | C) = 0`, making the whole document probability zero (because the independence assumption multiplies).

Laplace smoothing (add-one smoothing) fixes this:

```
P(word_j | C) = (count(word_j, C) + alpha) / (Sum_j count(word_j, C) + alpha * V)
```

where `V` is the vocabulary size and `alpha` is the smoothing parameter (default 1.0 in scikit-learn). Every word gets a small non-zero probability even if unseen. This is the single most important detail in Naive Bayes implementations.

Deeper Dive: Tree Pruning Mechanics

Cost-complexity pruning (ccp) is the canonical post-pruning algorithm. Define:

```
R_alpha(T) = R(T) + alpha * |T|
```

where `R(T)` is the tree's total leaf impurity and `|T|` is the number of leaves. Start from the fully grown tree `T_max`. Find the subtree that increases `R` least per leaf removed:

```
alpha_eff(subtree) = (R(pruned) - R(unpruned)) / (|pruned| - |unpruned|)
```

Prune the subtree with the smallest `alpha_eff`. Repeat. This produces a sequence of nested trees `T_max = T_0 > T_1 > ... > T_root`, each with a corresponding `alpha`. Use CV on `alpha` to pick the best tree in the sequence.

`DecisionTreeClassifier(ccp_alpha=...)` in scikit-learn exposes this.

Reflection Questions

1. You have a dataset with 10 features, 100 observations, and a binary target. Which model do you try first, and why?
2. A colleague suggests using KNN with `k = 1` for their credit scoring model ("it's the most faithful to the data"). Why is this a bad idea?
3. Why does feature subsampling in random forests reduce tree correlation?
4. Derive the OOB probability `(1 - 1/n)^n -> 1/e` by taking the logarithm and using `log(1 - x) ≈ -x`.
5. Why does Naive Bayes often produce well-ranked but poorly-calibrated probabilities?
6. What is the difference between pre-pruning (`max_depth`) and post-pruning (`ccp_alpha`)? When would you prefer each?

Lesson 3.4 — Gradient Boosting Deep Dive

Why This Matters

Between 2015 and 2020, XGBoost won more Kaggle competitions than every other algorithm combined. When LightGBM appeared, it matched XGBoost on accuracy while training five to ten times faster. When CatBoost appeared, it handled categorical features natively without the one-hot explosion that plagued the other two. Today, if you are building a tabular model and you are not trying at least one gradient boosting library, you are leaving accuracy on the table.

Gradient boosting is the dominant algorithm on tabular data. It deserves its own lesson not just because it is popular, but because understanding it unlocks a cluster of important ideas: second-order optimisation, regularised tree learning, histogram-based splitting, and the long-running debate between accuracy and speed.

This lesson covers:

- Why boosting reduces bias (whereas bagging reduces variance).
- The XGBoost second-order Taylor derivation, step by step.
- LightGBM's Gradient-based One-Side Sampling and histogram bins.
- CatBoost's ordered boosting and why it prevents target leakage.
- How to choose between the three libraries for a given problem.

Core Concepts

Boosting vs Bagging

Random Forests are a **bagging** method: build many high-variance models independently and average them. Averaging reduces variance because independent errors cancel. Bias is unchanged — the average of biased models is a biased model.

Boosting takes the opposite approach: build models **sequentially**, where each new model tries to correct the errors of the previous ones. The next model focuses on what the previous ones got wrong. This reduces bias, because each model adds capacity targeted at mistakes.

Formally, boosting builds an additive model:

```
F_M(x) = Sum_{m=1}^{M} gamma_m * h_m(x)
```

where each `h_m` is a weak learner (typically a shallow decision tree) and `gamma_m` is its contribution weight. Training is sequential: `h_{m+1}` is chosen to minimise the residual error of `F_m`.

AdaBoost (Historical Warmup)

AdaBoost (Freund and Schapire, 1995) was the first practical boosting algorithm. It works by reweighting training examples: misclassified points get higher weights, correctly classified points get lower weights, and the next tree is trained on the reweighted data. After all trees are trained, the final classifier is a weighted vote of the individual trees, with better trees getting heavier votes.

AdaBoost was revolutionary when it appeared but has been largely superseded by gradient boosting. The key insight it taught the field was that combining many weak learners could outperform a single strong learner.

Gradient Boosting: The General Framework

Gradient boosting (Friedman, 1999) frames boosting as **gradient descent in function space**. The idea: we want to minimise a loss `L(F)` over the space of functions `F`. At each step, compute the gradient of the loss with respect to the current model's predictions, and fit a weak learner to that gradient. Then update the model.

More concretely, suppose the current model is `F_{m-1}`. The loss is:

```
L = Sum_i l(y_i, F_{m-1}(x_i))
```

The "gradient" of the loss with respect to the predictions at point `i` is:

```
g_i = dl(y_i, F_{m-1}(x_i)) / dF_{m-1}(x_i)
```

For squared error loss, `l(y, F) = (1/2)(y - F)^2`, so `g_i = -(y_i - F_{m-1}(x_i))` — the negative residual. Gradient boosting on squared error reduces to fitting each new tree to the residuals of the current model. Elegantly simple.

The update rule is:

```
F_m(x) = F_{m-1}(x) - eta * h_m(x)
```

where `h_m` is a decision tree fit to the negative gradients, and `eta` is the learning rate (typically 0.01 to 0.1). Small `eta` with many trees usually outperforms large `eta` with few trees, because it allows finer corrections.

Mathematical Foundations: XGBoost's Second-Order Derivation

XGBoost (Chen and Guestrin, 2016) extends gradient boosting with a second-order Taylor expansion of the loss. This is the derivation that every ML engineer should have seen at least once.

The Objective

XGBoost's regularised objective is:

```
Obj = Sum_i l(y_i, F_{m-1}(x_i) + f_m(x_i)) + Omega(f_m) + (regularisation on earlier trees)
```

where `f_m` is the new tree being added. The regularisation term on the new tree is:

```
Omega(f) = gamma * T + (1/2) * lambda * Sum_{j=1}^{T} w_j^2
```

Here `T` is the number of leaves in the tree, `gamma` is a penalty per leaf, `w_j` is the weight (prediction) in leaf `j`, and `lambda` is the L2 penalty on leaf weights. This regularisation is the reason XGBoost is sometimes described as "regularised gradient boosting".

Second-Order Taylor Expansion

Taylor-expand the loss around the current prediction `F_{m-1}(x_i)`:

```
l(y_i, F_{m-1}(x_i) + f_m(x_i)) ≈ l(y_i, F_{m-1}(x_i))
                                  + g_i * f_m(x_i)
                                  + (1/2) * h_i * f_m(x_i)^2
```

where:

```
g_i = partial l(y_i, F_{m-1}(x_i)) / partial F_{m-1}(x_i)        [first derivative]
h_i = partial^2 l(y_i, F_{m-1}(x_i)) / partial F_{m-1}(x_i)^2    [second derivative]
```

Drop the constant term `l(y_i, F_{m-1}(x_i))` (it does not depend on `f_m`). The objective simplifies to:

```
Obj ≈ Sum_i [g_i * f_m(x_i) + (1/2) * h_i * f_m(x_i)^2] + Omega(f_m)
```

Optimal Leaf Weights

A tree partitions the input space into `T` disjoint leaves. Let `I_j` be the set of samples that fall into leaf `j`. In leaf `j`, all samples get the same prediction `w_j`. So the objective becomes:

```
Obj ≈ Sum_{j=1}^{T} [ (Sum_{i in I_j} g_i) * w_j + (1/2) * (Sum_{i in I_j} h_i) * w_j^2 ] + gamma * T + (1/2) * lambda * Sum_j w_j^2
```

Define `G_j = Sum_{i in I_j} g_i` and `H_j = Sum_{i in I_j} h_i`. Then:

```
Obj ≈ Sum_{j=1}^{T} [ G_j * w_j + (1/2) * (H_j + lambda) * w_j^2 ] + gamma * T
```

This is a quadratic in each `w_j`, and the quadratics are independent across leaves. Differentiate with respect to `w_j`:

```
dObj/dw_j = G_j + (H_j + lambda) * w_j = 0
```

Solve:

```
w_j^* = -G_j / (H_j + lambda)
```

This is the optimal leaf weight. Substituting back into the objective:

```
Obj^* = -(1/2) * Sum_{j=1}^{T} G_j^2 / (H_j + lambda) + gamma * T
```

Memorise this formula — it is the score XGBoost uses to evaluate a candidate tree structure.

The Split Gain Formula

Now consider splitting a leaf into a left child and a right child. Let `G_L`, `H_L` be the gradient/hessian sums of the left child, and `G_R`, `H_R` for the right child. Before the split, the parent had score:

```
-(1/2) * (G_L + G_R)^2 / (H_L + H_R + lambda)
```

After the split, the children have combined score:

```
-(1/2) * [G_L^2 / (H_L + lambda) + G_R^2 / (H_R + lambda)]
```

The gain from the split (reduction in objective, i.e. improvement) is:

```
Gain = (1/2) * [ G_L^2 / (H_L + lambda) + G_R^2 / (H_R + lambda) - (G_L + G_R)^2 / (H_L + H_R + lambda) ] - gamma
```

This is the XGBoost split gain formula. XGBoost scans all possible splits at every node and picks the one with the highest gain. If the best gain is negative, the split is rejected — this is how `gamma` prevents over-splitting. High `gamma` = fewer splits = smaller trees = more bias, less variance.

Notice the structure:

- `G_j^2 / (H_j + lambda)` measures how much "pull" each leaf has. Leaves with large absolute gradient (lots of error) and small hessian (model is uncertain) contribute more.
- `lambda` in the denominator shrinks the gain; it is the L2 regularisation on leaf weights.
- `gamma` is a flat tax per split; it controls tree size directly.

These two regularisers (`lambda` and `gamma`) are what make XGBoost more robust than plain gradient boosting.

Summary of XGBoost Hyperparameters

- `n_estimators`: number of boosting rounds (trees).
- `learning_rate` (eta): shrinkage per tree. Smaller means slower learning, usually better generalisation.
- `max_depth`: maximum tree depth. Controls complexity per tree.
- `min_child_weight`: minimum sum of hessians in a leaf. Prevents overfitting on noisy leaves.
- `reg_lambda`: L2 penalty on leaf weights (our `lambda`).
- `reg_alpha`: L1 penalty on leaf weights (Lasso-style sparsity in leaves).
- `gamma`: minimum loss reduction for a split (our `gamma`).
- `subsample`: fraction of rows used per tree (row subsampling).
- `colsample_bytree`: fraction of features used per tree.

LightGBM: Histograms and GOSS

LightGBM (Ke et al., 2017) is XGBoost's faster cousin. It introduces two key innovations.

Histogram-Based Split Finding

XGBoost's exact split finding considers every unique feature value as a candidate split point. For a feature with `n` unique values, that is `n - 1` candidates. Across `d` features and `n` samples, the cost per tree is `O(n * d)`.

LightGBM discretises each feature into a fixed number of bins (default 255). Instead of considering every unique value, it considers only the bin boundaries. This reduces the cost to `O(bins * d)`, which is constant in `n`. Histogram construction itself is `O(n)`, but it is a much simpler scan.

The approximation is remarkably tight: binning to 255 quantiles usually sacrifices less than 0.01 AUC, often none at all, while running five to ten times faster.

Gradient-Based One-Side Sampling (GOSS)

Boosting spends most of its time on samples the model has already learned well. Their gradients are small, so they barely contribute to split decisions, but they still cost computation. GOSS asks: why not skip them?

The GOSS recipe:

1. Sort samples by absolute gradient in descending order.
2. Keep the top `a%` (e.g., 20%) — these are the "important" samples with large gradients.
3. Randomly sample `b%` (e.g., 10%) of the remaining samples.
4. When computing split gains, upweight the small-gradient samples by a factor of `(1 - a) / b` to compensate for subsampling.

This preserves the expected value of the split gain while using only `a + b` fraction of the data. Typical settings (a = 0.2, b = 0.1) cut the data used per split to 30% with minimal accuracy loss.

The upweighting factor `(1 - a) / b` is crucial. It ensures that the expected sum of gradients in the sampled set equals the sum in the full set, making the split gains unbiased estimators.

Leaf-Wise Tree Growth

Most tree learners grow depth-wise: expand all nodes at depth `d` before moving to depth `d + 1`. LightGBM grows leaf-wise: always expand the leaf whose split gives the largest gain. This produces deeper, more unbalanced trees with the same number of leaves, and usually better accuracy. It is also more prone to overfitting on small datasets, so set `num_leaves` conservatively (e.g., 31–127 for most tabular problems).

CatBoost: Ordered Boosting and Native Categorical Handling

CatBoost (Prokhorenkova et al., 2018) focuses on two problems: target leakage in standard gradient boosting and the inefficiency of one-hot encoding for high-cardinality categoricals.

Target Leakage in Standard Boosting

Consider computing gradients. For sample `i`, the gradient `g_i` depends on the current model's prediction for `i`, which depends on the trees trained so far, which were trained on samples including `i`. So `g_i` is not an unbiased estimate — the model has seen the target of sample `i` when computing it.

This leakage is subtle but causes real overfitting, especially when categorical encoding is involved. Target encoding (replacing a category with its mean target value) is a particularly bad offender: the target encoding of `zipcode = 048622` uses the labels of other `048622` samples, but those other samples are the same rows being used to compute gradients.

Ordered Boosting

CatBoost's fix is elegant. Order the samples randomly. When computing the gradient for sample `i`, use only samples `1, ..., i-1` to train the auxiliary model that computes the gradient. This guarantees that sample `i`'s own target never influences its own gradient. Similarly, target encodings for sample `i` are computed from samples earlier in the ordering.

The cost is that you need multiple orderings to get stable gradients (CatBoost uses several "permutation trees"), but the result is a model that is provably resistant to target leakage.

Native Categorical Handling

Rather than one-hot encoding a category with 10,000 levels into 10,000 columns, CatBoost stores the category directly and uses ordered target statistics as the feature value. Combined with ordered boosting, this avoids the leakage that normally plagues target encoding.

For high-cardinality categorical features (zip codes, SKUs, user IDs), CatBoost is often the best choice out of the box.

When to Choose Which Library

- **XGBoost**: the standard. When you want maximum flexibility, the best documentation, and the library most likely to be supported everywhere. Slightly slower than LightGBM.
- **LightGBM**: when training speed matters. Large datasets. Iterative experimentation. Pair with careful `num_leaves` tuning.
- **CatBoost**: when you have high-cardinality categoricals and cannot afford manual encoding. When you want protection from target leakage.

On most benchmarks, the three are within 1 AUC point of each other when properly tuned. The choice is about engineering ergonomics, not accuracy.

Kailash Engine: TrainingPipeline and AutoMLEngine

Kailash's `AutoMLEngine` wraps all three gradient boosting libraries behind a single interface, handling cross-validation, early stopping, and hyperparameter search. For simple use cases, `TrainingPipeline` is enough.

```
from kailash_ml import TrainingPipeline
import lightgbm as lgb

pipeline = TrainingPipeline(
    model=lgb.LGBMClassifier(
        n_estimators=500,
        learning_rate=0.05,
        num_leaves=31,
        subsample=0.8,
        colsample_bytree=0.8,
        reg_lambda=1.0,
        reg_alpha=0.0,
        random_state=42,
    ),
    cv_strategy="stratified_kfold",
    cv_folds=5,
    scoring=["roc_auc", "average_precision", "log_loss"],
)

result = pipeline.fit(X_train, y_train)
```

For more automation, `AutoMLEngine` will iterate over multiple model families and hyperparameters:

```
from kailash_ml import AutoMLEngine

automl = AutoMLEngine(
    model_families=["xgboost", "lightgbm", "catboost"],
    metric="average_precision",
    time_budget_seconds=600,
)
best = automl.fit(X_train, y_train)
print(f"Best family: {best.family}, AP = {best.score:.3f}")
```

Worked Example: Credit Scoring with Three Boosters

Following MLFP03 ex_4, we train XGBoost, LightGBM, and CatBoost on the Singapore credit scoring dataset and compare their performance.

```
import numpy as np
import polars as pl
import time
import xgboost as xgb
import lightgbm as lgb
from catboost import CatBoostClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import (
    average_precision_score, roc_auc_score, log_loss, brier_score_loss,
)
from shared import MLFPDataLoader

loader = MLFPDataLoader()
credit = loader.load("mlfp02", "credit_scoring.parquet")

feature_cols = [c for c in credit.columns if c != "default"]
X = credit.select(feature_cols).drop_nulls().to_numpy()
y = credit.select("default").to_series().to_numpy()[:len(X)]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Train XGBoost:

```
start = time.perf_counter()
xgb_model = xgb.XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    reg_lambda=1.0,
    reg_alpha=0.0,
    gamma=0.0,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="aucpr",
    random_state=42,
)
xgb_model.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
xgb_time = time.perf_counter() - start

p_xgb = xgb_model.predict_proba(X_test)[:, 1]
print(f"XGBoost:   AP = {average_precision_score(y_test, p_xgb):.3f}  "
      f"AUC = {roc_auc_score(y_test, p_xgb):.3f}  "
      f"LogLoss = {log_loss(y_test, p_xgb):.3f}  "
      f"time = {xgb_time:.1f}s")
```

Train LightGBM:

```
start = time.perf_counter()
lgb_model = lgb.LGBMClassifier(
    n_estimators=500,
    learning_rate=0.05,
    num_leaves=31,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,
    random_state=42,
)
lgb_model.fit(X_train, y_train, eval_set=[(X_test, y_test)])
lgb_time = time.perf_counter() - start

p_lgb = lgb_model.predict_proba(X_test)[:, 1]
print(f"LightGBM:  AP = {average_precision_score(y_test, p_lgb):.3f}  "
      f"AUC = {roc_auc_score(y_test, p_lgb):.3f}  "
      f"time = {lgb_time:.1f}s")
```

Train CatBoost:

```
start = time.perf_counter()
cat_model = CatBoostClassifier(
    iterations=500,
    learning_rate=0.05,
    depth=6,
    l2_leaf_reg=1.0,
    random_seed=42,
    verbose=False,
)
cat_model.fit(X_train, y_train)
cat_time = time.perf_counter() - start

p_cat = cat_model.predict_proba(X_test)[:, 1]
print(f"CatBoost:  AP = {average_precision_score(y_test, p_cat):.3f}  "
      f"AUC = {roc_auc_score(y_test, p_cat):.3f}  "
      f"time = {cat_time:.1f}s")
```

Typical results on the credit dataset: all three achieve AP between 0.55 and 0.62 (the baseline rate is 0.12, so these are substantial lifts). LightGBM is usually the fastest. CatBoost is slowest on numeric-only features but wins when categoricals dominate.

Computing Split Gain by Hand

Suppose a node has 10 samples with gradients:

```
g = [-0.8, -0.7, -0.6, -0.5, 0.1, 0.2, 0.4, 0.5, 0.6, 0.8]
h = [ 0.2,  0.2,  0.2,  0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2]
```

With `lambda = 1.0`, `gamma = 0.1`.

Parent: `G = Sum(g) = 0.0`, `H = Sum(h) = 2.0`. Score component: `0.0^2 / (2.0 + 1.0) = 0`.

Consider splitting into left (first 5 samples) and right (last 5):

- Left: `G_L = -0.8 - 0.7 - 0.6 - 0.5 + 0.1 = -2.5`, `H_L = 1.0`. Term: `(-2.5)^2 / (1.0 + 1.0) = 6.25 / 2.0 = 3.125`.
- Right: `G_R = 0.2 + 0.4 + 0.5 + 0.6 + 0.8 = 2.5`, `H_R = 1.0`. Term: `(2.5)^2 / 2.0 = 3.125`.

```
Gain = (1/2) * [3.125 + 3.125 - 0] - 0.1
     = (1/2) * 6.25 - 0.1
     = 3.125 - 0.1
     = 3.025
```

Positive and large — the split is clearly worth taking. The optimal leaf weights are:

```
w_L^* = -G_L / (H_L + lambda) = 2.5 / 2.0 = 1.25
w_R^* = -G_R / (H_R + lambda) = -2.5 / 2.0 = -1.25
```

These are the raw score contributions each leaf adds to its samples. The actual tree output passes through a shrinkage of `eta` per round.

Try It Yourself

Exercise A (easy). Train a LightGBM classifier with `num_leaves in {7, 15, 31, 63, 127, 255}` on the credit data. Plot validation AUC versus `num_leaves`. Which value overfits? Which underfits?

Exercise B (medium). Pick one feature from the credit dataset and build a partial dependence curve by hand: sweep the feature across 20 values, for each value set the feature in the entire validation set to that value and record the mean predicted probability. Plot. Interpret the shape.

Exercise C (hard). Implement a minimal gradient booster. Using scikit-learn's `DecisionTreeRegressor(max_depth=3)` as the base learner, write a loop that fits 100 trees sequentially, each on the negative gradient of squared error loss. Compare the resulting predictions with scikit-learn's `GradientBoostingRegressor` with the same settings. They should agree to high precision.

Cross-References

- **Lesson 3.2** Regularisation — the `lambda` and `gamma` here are the same idea.
- **Lesson 3.3** Decision Trees and Random Forests — boosting uses trees as base learners.
- **Forward link:** Lesson 3.5 will calibrate these models' probabilities with Platt scaling.
- **Forward link:** Lesson 3.6 uses TreeSHAP to explain boosted tree predictions efficiently.
- **Forward link:** Lesson 3.7 will tune these models with Bayesian optimisation.

Deeper Dive: Learning Rate and Shrinkage

Boosting updates `F_m = F_{m-1} + eta * f_m(x)`. The learning rate `eta` (typically 0.01 to 0.1) shrinks each tree's contribution. Why not just set `eta = 1`?

Small `eta` means each tree corrects only a fraction of the residual. The next tree still sees substantial error. More trees are needed, but the final model is more robust because no single tree dominates. Think of `eta` as a regulariser: it prevents the model from over-committing to any one direction too quickly.

Empirically, `eta = 0.05` with `n_estimators = 500` nearly always outperforms `eta = 0.5` with `n_estimators = 50`. The rule of thumb: halve `eta`, double `n_estimators`. Stop when validation loss no longer improves (early stopping).

Deeper Dive: Early Stopping

Training with a fixed `n_estimators` is wasteful — usually the validation loss bottoms out long before the last tree is built. Early stopping monitors validation loss after each tree and stops when it has not improved for `k` consecutive rounds (typically 50).

```
import lightgbm as lgb

model = lgb.LGBMClassifier(n_estimators=5000, learning_rate=0.02, random_state=42)
model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    callbacks=[lgb.early_stopping(stopping_rounds=50)],
)
print(f"Best iteration: {model.best_iteration_}")
```

Early stopping is the single most important trick in gradient boosting production code. It prevents both under-training (too few trees) and over-training (memorisation).

Deeper Dive: Understanding Gradients and Hessians by Loss

The XGBoost derivation used generic `g_i` and `h_i` — first and second derivatives of the loss. For common losses:

Squared error `L(y, F) = (1/2)(y - F)^2`: - `g = F - y` (negative residual) - `h = 1`

Logistic loss (binary classification with `F = logit`): `L = -y log(sigmoid(F)) - (1 - y) log(1 - sigmoid(F))`: - `g = sigmoid(F) - y = p - y` - `h = p(1 - p)`

Multiclass log loss (K classes): softmax and multiclass cross-entropy give per-class gradients and hessians.

Huber loss (robust regression): behaves like squared error for small errors and like absolute error for large errors, bounding the influence of outliers.

The beauty of XGBoost's formulation is that you can plug in any twice-differentiable loss function and the machinery works. Custom losses (focal loss, ranking loss) are implemented this way.

Reflection Questions

1. Why does boosting reduce bias whereas bagging reduces variance?
2. In the split gain formula, what happens if `lambda -> infinity`? What about `gamma -> infinity`?
3. GOSS uses an upweighting factor `(1 - a) / b`. Derive why this specific factor preserves the expected sum of gradients.
4. You train XGBoost with `n_estimators = 10000` and no early stopping. Your validation AUC is worse than with `n_estimators = 500`. What is happening and how do you fix it?
5. Write the gradient and hessian for logistic loss, starting from `L = -y log p - (1 - y) log(1 - p)` with `p = sigmoid(F)`.
6. CatBoost's ordered boosting costs more memory and training time. When is the extra cost justified?

Lesson 3.5 — Model Evaluation, Imbalance, and Calibration

Why This Matters

A Singapore bank deployed a fraud detection model in 2020. The model was reported to have **99.6% accuracy**. Management was delighted. The model went live. A week later, the fraud team called: "We haven't caught a single fraudster."

The base rate of fraud in the dataset was 0.4%. A model that predicted "no fraud" for every transaction would achieve 99.6% accuracy — exactly what the model was doing. Accuracy is a catastrophic metric for imbalanced problems because the majority class dominates the score. The bank swapped to area under the precision-recall curve (AUC-PR). Their 99.6% accuracy model's AUC-PR was 0.09 — worse than random for the problem that actually mattered.

The bank's second lesson was calibration. They asked the model for "high-risk" transactions (predicted probability > 0.8) and expected 80% of those to actually be fraud. In reality, only 34% were. The model ranked transactions correctly but its probabilities were meaningless. They learned about **Platt scaling** and recalibrated the model. After calibration, a transaction with `p = 0.8` was fraud about 78% of the time. The business could finally set thresholds based on probabilities.

This lesson covers:

- The complete metrics taxonomy for classification and regression.
- Why accuracy fails on imbalanced data, and what to use instead.
- Class imbalance solutions: SMOTE, cost-sensitive learning, focal loss.
- Probability calibration: Platt scaling, isotonic regression, Brier score.
- How to read a calibration plot.

Core Concepts

The Complete Metrics Taxonomy

Classification metrics.

Start from the confusion matrix. For binary classification:

```
                Predicted Positive  Predicted Negative
Actual Positive        TP                  FN
Actual Negative        FP                  TN
```

From these four numbers, derive:

- **Accuracy** = `(TP + TN) / (TP + FP + TN + FN)`. Fraction correct. Useless on imbalanced data.
- **Precision** = `TP / (TP + FP)`. Of the items predicted positive, what fraction actually are positive. Answer to: "when the model says yes, how often is it right?"
- **Recall** (sensitivity, true positive rate) = `TP / (TP + FN)`. Of the actual positives, what fraction did we catch. Answer to: "of all the positives, how many did we find?"
- **Specificity** (true negative rate) = `TN / (TN + FP)`. Of the actual negatives, what fraction did we correctly label negative.
- **F1 score** = `2 * (precision * recall) / (precision + recall)`. Harmonic mean of precision and recall.
- **False positive rate** (FPR) = `FP / (FP + TN)` = `1 - specificity`.

Threshold-dependent vs threshold-independent metrics. All of the above depend on a threshold (default 0.5). A single confusion matrix is a snapshot at one threshold. Threshold-independent metrics let you evaluate the classifier's ranking quality without committing to a threshold.

- **ROC-AUC**: area under the ROC curve (TPR vs FPR as threshold varies). Interpretation: the probability that a random positive has a higher predicted probability than a random negative. Range 0.5 (random) to 1.0 (perfect).
- **AUC-PR (average precision)**: area under the precision-recall curve. More informative than ROC-AUC on imbalanced data because it does not reward true negatives.
- **Log loss** (cross-entropy): `-1/N * Sum_i [y_i log(p_i) + (1 - y_i) log(1 - p_i)]`. Penalises confident wrong predictions heavily. The proper scoring rule for probabilistic classifiers.

When to use which.

Situation	Metric
Balanced classes	Accuracy or F1
Imbalanced classes	AUC-PR, F1
Ranking matters (any threshold)	ROC-AUC or AUC-PR
Probabilistic output needed	Log loss, Brier score
Business cost differs	Custom cost matrix

Regression metrics.

- **R-squared**: fraction of variance explained. Range (–infinity, 1]. Beware: can be misleading when comparing across datasets.
- **Adjusted R-squared**: penalises extra features. `1 - (1 - R^2) * (n - 1) / (n - p - 1)`.
- **MAE** (mean absolute error): `1/N * Sum |y - y_hat|`. Robust to outliers.
- **MSE** (mean squared error): `1/N * Sum (y - y_hat)^2`. Penalises large errors more.
- **RMSE** (root mean squared error): `sqrt(MSE)`. Same units as the target.
- **MAPE** (mean absolute percentage error): `1/N * Sum |y - y_hat| / |y|`. Scale-free but undefined when `y = 0`.

Why Accuracy Fails on Imbalanced Data

Suppose 99% of your transactions are legitimate and 1% are fraudulent. A constant-predictor that says "legit" to everything has 99% accuracy and zero business value. The problem is that accuracy weights every observation equally, so the majority class dominates.

Precision and recall fix this by focusing on the positive class:

- Precision asks: when I raise an alarm, am I usually right?
- Recall asks: when there is a fraud, do I catch it?

For fraud, you care deeply about recall — missing a fraud is expensive. You also care about precision — every alarm triggers a manual review that costs time. F1 balances both. AUC-PR summarises the precision-recall trade-off across all thresholds.

Class Imbalance Solutions

SMOTE (Synthetic Minority Over-sampling Technique). For each minority-class point, find its `k` nearest minority-class neighbours. Generate a new synthetic point by interpolating between the point and a randomly chosen neighbour. Repeat until the classes are balanced.

SMOTE often helps when the minority class has enough samples to form clusters (say, 500+), and the problem is that there are too few relative to the majority. It often hurts when:

- The minority class is very small (under 100). Neighbours are unreliable.
- The data is high-dimensional. Interpolation in high-D produces unrealistic samples.
- The minority class has high variance or multimodal structure. Interpolation smears the modes.
- The boundary between classes is noisy. SMOTE amplifies noise near the boundary.

Treat SMOTE as one tool in the kit, not as a silver bullet.

Cost-sensitive learning. Instead of balancing the data, weight the loss function. Multiply the loss of each minority-class sample by a factor (typically `n_majority / n_minority`). Most scikit-learn models accept a `class_weight="balanced"` or `sample_weight` parameter that does this directly. Gradient boosting libraries have `scale_pos_weight` (XGBoost) or `is_unbalance` (LightGBM).

Cost-sensitive learning is often a better choice than SMOTE because it does not create synthetic samples — it just tells the model that mistakes on the minority class are expensive. The model adjusts its decision boundary accordingly.

Business cost matrix. Sometimes the cost of false positives and false negatives is known explicitly. For the Singapore credit scoring case in MLFP03 ex_5, the cost matrix is:

Actual	Predicted default	Predicted good
Actual default	cost = 0 (blocked)	$10,000 (loss)
Actual good	$100 (lost biz)	cost = 0

The optimal threshold is then chosen to minimise expected cost, not to maximise F1 or accuracy. The math:

```
Expected cost(t) = FN_cost * P(y=1) * (1 - recall(t))
                 + FP_cost * P(y=0) * (1 - specificity(t))
```

Sweep the threshold `t`, compute expected cost, pick the `t` that minimises it. For a 100:1 cost ratio, the optimal threshold is usually well below 0.5 — you want to raise alarms on suspicious borderline cases because missing a real default costs far more than a false alarm.

Focal Loss

Cross-entropy loss treats all errors equally. But for heavy imbalance, most training examples are easy negatives — the model is already 99% confident they are negative, and the loss contribution is tiny. The occasional hard example gets lost in the sea of easy ones. Focal loss down-weights the easy examples so the model focuses on hard ones.

Starting from binary cross-entropy with `p_t` the predicted probability of the true class:

```
CE(p_t) = -log(p_t)
```

Focal loss multiplies by a modulating factor `(1 - p_t)^gamma`:

```
FL(p_t) = -(1 - p_t)^gamma * log(p_t)
```

When `p_t` is close to 1 (easy example), `(1 - p_t)^gamma` is near zero, so the loss is down-weighted. When `p_t` is small (hard example), the factor is near 1 and the loss is preserved. The hyperparameter `gamma` controls the intensity; `gamma = 0` recovers cross-entropy, `gamma = 2` is the default suggested by Lin et al. (2017).

Often an additional class weight `alpha_t` is added to handle imbalance directly:

```
FL(p_t) = -alpha_t * (1 - p_t)^gamma * log(p_t)
```

This is the full form used in RetinaNet and many modern detection systems.

Deriving the gradient. At its simplest, focal loss reshapes the loss surface. Let `p = sigmoid(z)`. The focal loss gradient with respect to the logit `z` is:

```
dFL/dz = (1 - p)^gamma * (gamma * p * log(p) + p - y)
```

Compared to cross-entropy's gradient `p - y`, focal loss scales the gradient by `(1 - p)^gamma`, shrinking easy-example gradients. Hard examples — where `p` is far from `y` — retain most of their gradient.

Probability Calibration

A classifier is **calibrated** if, among the instances it assigns probability `p`, the actual positive rate is approximately `p`. Formally:

```
P(y = 1 | p_hat = p) ≈ p  for all p in [0, 1]
```

Why does this matter? Because decisions depend on probabilities, not rankings. If you are deciding whether to offer a loan based on a 10% default threshold, you need the model's 10% to actually mean 10% — otherwise you are rejecting the wrong people.

Most models are not naturally calibrated:

- **Random forests** are poorly calibrated because they vote among trees; the proportion of trees voting yes is not a probability estimate.
- **SVMs** produce scores, not probabilities. Raw scores need to be mapped to probabilities.
- **Neural networks** tend to be over-confident — they assign probabilities too close to 0 or 1.
- **Boosted trees** are often reasonably calibrated but benefit from post-hoc adjustment.

Brier score measures calibration plus accuracy:

```
BS = (1/N) * Sum_i (p_i - y_i)^2
```

A lower Brier score is better. A perfectly calibrated model has Brier score equal to `p * (1 - p)` where `p` is the base rate. The Brier score can be decomposed into calibration, resolution, and uncertainty components (Murphy, 1973), but the headline number is enough for most practical purposes.

Platt Scaling

Platt (1999) proposed the simplest calibration method: fit a logistic regression on the model's output scores.

Let `s_i` be the model's raw score for observation `i`. Fit:

```
p_i = 1 / (1 + exp(A * s_i + B))
```

on a held-out calibration set, using the true labels. This is a one-dimensional logistic regression with two parameters `A` and `B`.

Platt scaling works well when the miscalibration follows a sigmoid shape — which is the case for SVMs and often for boosted trees on moderate-to-large datasets. It fails when the miscalibration is non-monotonic or has a different functional form.

Isotonic Regression

Isotonic regression is a non-parametric alternative. It fits any monotonic function from scores to probabilities:

```
p_i = PAV(s_i)
```

where PAV is the pool-adjacent-violators algorithm (an `O(n)` algorithm for isotonic regression). It can capture any monotonic miscalibration but needs more data (500+ calibration samples) to avoid overfitting.

Rule of thumb:

- Small calibration set (< 1000): use Platt.
- Large calibration set and non-sigmoidal miscalibration: use isotonic.
- Always evaluate with Brier score on a separate held-out set.

Reading a Calibration Plot

A calibration plot (reliability diagram) bins the predicted probabilities into deciles, then for each bin plots:

- X-axis: mean predicted probability in the bin.
- Y-axis: fraction of positives in the bin.

A perfectly calibrated model lies on the diagonal `y = x`. Deviations tell you how miscalibrated the model is:

- **Above the diagonal**: model under-predicts. It says 30% but 45% are actually positive.
- **Below the diagonal**: model over-predicts. It says 70% but only 50% are actually positive.

An S-shape (low bins below the diagonal, high bins above) indicates over-confidence — the model pushes probabilities too far toward 0 and 1. This is typical of neural networks and boosted trees. The fix is Platt scaling, which is an inverse sigmoid.

Kailash Engine: Metrics, Sampling, and Calibration

```
from kailash_ml import TrainingPipeline
from sklearn.calibration import CalibratedClassifierCV
import lightgbm as lgb

base = lgb.LGBMClassifier(class_weight="balanced", random_state=42)
calibrated = CalibratedClassifierCV(base, method="sigmoid", cv=5)

pipeline = TrainingPipeline(
    model=calibrated,
    cv_strategy="stratified_kfold",
    cv_folds=5,
    scoring=["average_precision", "brier_score", "roc_auc"],
)
result = pipeline.fit(X_train, y_train)
```

`CalibratedClassifierCV(method="sigmoid")` is Platt scaling. `method="isotonic"` is isotonic regression. `cv=5` means the calibration is done via 5-fold CV to avoid using training data twice.

Worked Example: Imbalanced Credit Scoring

Following MLFP03 ex_5:

```
import numpy as np
import polars as pl
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    average_precision_score, roc_auc_score, brier_score_loss,
    precision_recall_curve, f1_score,
)
from sklearn.calibration import CalibratedClassifierCV, calibration_curve
from imblearn.over_sampling import SMOTE
import lightgbm as lgb

from shared import MLFPDataLoader
loader = MLFPDataLoader()
credit = loader.load("mlfp02", "credit_scoring.parquet")
```

```
X = credit.drop("default").to_numpy()
y = credit.select("default").to_series().to_numpy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42,
)

print(f"Train positive rate: {y_train.mean():.3f}")
print(f"Test positive rate: {y_test.mean():.3f}")
```

Baseline (no imbalance handling):

```
base = lgb.LGBMClassifier(random_state=42)
base.fit(X_train, y_train)
p_base = base.predict_proba(X_test)[:, 1]

print(f"Baseline AP:       {average_precision_score(y_test, p_base):.3f}")
print(f"Baseline ROC-AUC:  {roc_auc_score(y_test, p_base):.3f}")
print(f"Baseline Brier:    {brier_score_loss(y_test, p_base):.4f}")
print(f"Baseline accuracy: {(p_base > 0.5).astype(int).mean():.3f}")
```

SMOTE:

```
smote = SMOTE(random_state=42)
X_sm, y_sm = smote.fit_resample(X_train, y_train)

smote_model = lgb.LGBMClassifier(random_state=42)
smote_model.fit(X_sm, y_sm)
p_smote = smote_model.predict_proba(X_test)[:, 1]
print(f"SMOTE AP: {average_precision_score(y_test, p_smote):.3f}")
```

Notice: SMOTE often improves AP slightly but **ruins calibration**. The Brier score after SMOTE is usually worse than the baseline because the model is trained on synthetic class proportions that do not match deployment.

Cost-sensitive learning:

```
cs_model = lgb.LGBMClassifier(scale_pos_weight=y_train.sum() / (len(y_train) - y_train.sum()),
                              random_state=42)
cs_model.fit(X_train, y_train)
p_cs = cs_model.predict_proba(X_test)[:, 1]
print(f"Cost-sensitive AP: {average_precision_score(y_test, p_cs):.3f}")
```

Threshold optimisation from cost matrix:

```
fp_cost = 100
fn_cost = 10000

thresholds = np.linspace(0.01, 0.99, 99)
expected_costs = []
for t in thresholds:
    preds = (p_cs >= t).astype(int)
    fn = np.sum((preds == 0) & (y_test == 1))
    fp = np.sum((preds == 1) & (y_test == 0))
    expected_costs.append(fn * fn_cost + fp * fp_cost)

optimal_t = thresholds[np.argmin(expected_costs)]
print(f"Optimal threshold: {optimal_t:.2f}")
print(f"Minimum cost: ${min(expected_costs):,.0f}")
```

For a 100:1 cost ratio, the optimal threshold is usually around 0.15–0.25, much lower than the default 0.5.

Post-hoc calibration (Platt scaling):

```
cal_model = CalibratedClassifierCV(lgb.LGBMClassifier(random_state=42),
                                   method="sigmoid", cv=5)
cal_model.fit(X_train, y_train)
p_cal = cal_model.predict_proba(X_test)[:, 1]
print(f"Calibrated Brier: {brier_score_loss(y_test, p_cal):.4f}")

# Calibration curve
frac_pos, mean_pred = calibration_curve(y_test, p_cal, n_bins=10)
for mp, fp in zip(mean_pred, frac_pos):
    print(f"  pred={mp:.2f} -> actual={fp:.2f}")
```

After Platt scaling, the Brier score should improve by 5–15% relative to the uncalibrated baseline, and the calibration curve should lie closer to the diagonal.

Try It Yourself

Exercise A (easy). Plot the reliability diagram for both the uncalibrated and calibrated models on the credit dataset. Which one lies closer to the diagonal?

Exercise B (medium). Sweep `fp_cost` from $10 to $10,000 (keeping `fn_cost = $10,000`). For each cost ratio, compute the optimal threshold and the expected cost. Plot the optimal threshold vs the cost ratio. What shape is the curve?

Exercise C (hard). Implement focal loss as a custom LightGBM objective function. Train with `gamma in {0, 0.5, 1, 2, 5}` and compare to cross-entropy. Does focal loss improve AP on this dataset?

Cross-References

- **Module 2, Lesson 2.2** Sampling and the role of the sample mean — appears here as base-rate in the cost analysis.
- **Lesson 3.2** Bias-variance — over-confidence in predictions is a variance symptom.
- **Lesson 3.4** Gradient boosting — the models we are calibrating.
- **Forward link:** Lesson 3.8 will use the Brier score as a production monitoring signal.

Deeper Dive: ROC vs PR Curves on Imbalanced Data

ROC curves plot TPR (true positive rate) against FPR (false positive rate). Precision-Recall curves plot precision against recall. Both are threshold-sweeps over the classifier's scores.

For a rare-positive problem, ROC is misleading. FPR = FP / (FP + TN). The denominator is dominated by true negatives (the vast majority of samples). Adding even many false positives barely moves FPR. So the ROC curve can look great while precision is terrible.

PR curves do not have this problem. Precision = TP / (TP + FP). Adding false positives directly hurts precision. If your model is predicting lots of positives that are actually negatives, PR will show it immediately.

Rule of thumb: if the positive class is below 10%, prefer AUC-PR (average precision) over ROC-AUC. Above 10%, both are fine.

Deeper Dive: Isotonic Regression by Pool-Adjacent-Violators

Isotonic regression fits a monotonically non-decreasing function to (score, label) pairs. The algorithm:

1. Sort samples by score ascending: `s_1 <= s_2 <= ... <= s_n`.
2. Initialise `p_i = y_i` for each.
3. Walk left to right. Whenever `p_i > p_{i+1}`, merge them into a block with `p = (p_i + p_{i+1}) / 2`. If merging causes a new violation with the block to the left, merge again.
4. Continue until no violations remain.

The result is a step function, non-decreasing, that minimises the squared error between calibrated probabilities and labels. PAV runs in `O(n)`.

Isotonic regression needs enough data per "plateau" to be reliable. With 500+ calibration samples, isotonic beats Platt. With fewer, the flexibility overfits and Platt wins.

Deeper Dive: The Cost-Optimal Threshold, Derived

Given costs `c_FN` (false negative) and `c_FP` (false positive), and the classifier's predicted probability `p`, the optimal decision is to predict positive if the expected cost of predicting positive is lower than the expected cost of predicting negative:

```
c_FP * (1 - p) < c_FN * p
```

Solving:

```
p > c_FP / (c_FP + c_FN)
```

For `c_FP = 100` and `c_FN = 10000`, the optimal threshold is `100 / 10100 = 0.0099`. Anyone with predicted default probability above 1% should be flagged. This is a much lower threshold than the default 0.5, and it is the *principled* answer given the cost matrix.

Of course, this assumes the classifier is calibrated. If the classifier is over-confident (probabilities too close to 0 or 1), the threshold derived from the cost matrix will be wrong. This is why calibration and thresholding are inseparable.

Reflection Questions

1. Your model has ROC-AUC 0.95 but AP of 0.12 on a dataset with 0.5% positive rate. Is this model useful?
2. When is SMOTE helpful? When does it hurt? Give a specific example of each.
3. Why does Platt scaling use a logistic regression rather than a linear regression on model outputs?
4. Your stakeholder demands "90% accuracy or we can't ship". The base rate is 3%. What do you say?
5. Derive the optimal threshold formula `p > c_FP / (c_FP + c_FN)` from the expected cost minimisation.
6. You switch from cross-entropy to focal loss with `gamma = 2`. Validation AP improves by 0.02 but log loss gets worse. What is happening?

Lesson 3.6 — Interpretability and Fairness

Why This Matters

In November 2019, a major credit card company faced public outcry when users discovered that women were systematically given lower credit limits than men with equivalent financial profiles — including cases where married couples with shared finances received dramatically different limits. When asked to explain,

the company said: “The algorithm is proprietary, and none of our engineers can explain individual decisions.” Within a week, the New York Department of Financial Services opened an investigation. The brand damage was severe.

The problem was not just the disparity (though that was serious). It was the absence of an explanation. A model that cannot justify its decisions cannot be defended, corrected, or trusted. For consumer-facing ML — credit, hiring, healthcare, criminal justice — explainability is no longer optional. In Singapore, the Monetary Authority (MAS) requires that credit decisions affecting consumers be explainable under the Code of Consumer Banking Practice. The Personal Data Protection Act (PDPA) gives individuals the right to request an explanation of automated decisions that significantly affect them.

This lesson gives you two tools for explainability (SHAP and LIME) and one framework for fairness (the impossibility theorem). You will learn:

- How Shapley values allocate a prediction’s total value to its input features.
- The four axioms that uniquely determine Shapley values.
- How TreeSHAP exploits tree structure for efficient computation.
- LIME as a model-agnostic alternative.
- The impossibility theorem: why you cannot simultaneously satisfy all fairness criteria.
- How to conduct a fairness audit.

Core Concepts

The Explanation Problem

For a random forest with 500 trees, how do you explain why it predicted a 73% default probability for one specific customer? You cannot read 500 trees. You need a summary that answers: “which features drove this prediction, and by how much?”

Two flavours of explanation:

1. **Global explanations**: what matters on average across the whole dataset. “Income is the most important feature overall.”
2. **Local explanations**: what mattered for a single prediction. “For this customer, the model predicted 73% because of their $50k debt and only 2 years employment history.”

Global explanations help you understand the model; local explanations help you defend individual decisions. SHAP gives you both.

Shapley Values: The Intuition

Shapley values come from cooperative game theory (Shapley, 1953). Imagine three workers building a house together. They can produce 100 dollars of value. How do you split the 100 fairly among them?

The Shapley value for worker `i` is: average over all possible orderings of the workers, the marginal contribution of `i` when they join the coalition formed by those ordered before them. Concretely, if worker 1 alone produces 40, worker 2 alone produces 50, worker 3 alone produces 20, and all together produce 100:

- In the ordering (1, 2, 3): 1 contributes 40, 2 contributes (1+2)–(1) = let’s say 70–40 = 30, 3 contributes 100–70 = 30.
- In each of the 6 possible orderings, compute each worker’s marginal contribution.
- Average across orderings.

The resulting average is the Shapley value. It satisfies four properties that capture “fairness” in an axiomatic sense.

The Four Shapley Axioms

1. **Efficiency** (additivity of contributions). The Shapley values sum to the total value: `Sum_i phi_i = v(all) - v(none)`. For an ML prediction, this means the SHAP values sum to the difference between the prediction and the average prediction (the “baseline”).

2. **Symmetry**. If two features `i` and `j` contribute equally to every possible coalition (i.e., `v(S ∪ {i}) = v(S ∪ {j})` for all `S`), they get equal Shapley values.

3. **Dummy** (null player). If a feature contributes nothing to any coalition, its Shapley value is zero.

4. **Linearity** (additivity over games). If you have two prediction problems with values `v_1` and `v_2`, the Shapley value for the combined game is the sum: `phi_i(v_1 + v_2) = phi_i(v_1) + phi_i(v_2)`.

Shapley (1953) proved: **these four axioms uniquely determine the Shapley value**. Any other allocation method violates at least one. That is the foundation of SHAP’s claim to be “principled”.

The Shapley Value Formula

Let `F` be the set of all features, `S` a subset of features (excluding `i`), and `f(S)` the model’s prediction using only features in `S` (with the others marginalised out somehow). The Shapley value of feature `i` is:

```
phi_i = Sum_{S in F \ {i}} [|S|! * (|F| - |S| - 1)! / |F|!] * [f(S ∪ {i}) - f(S)]
```

The sum is over all subsets `S` not containing `i`. The weight is the combinatorial term that counts orderings: in how many orderings does feature `i` join after exactly the features in `S`? Dividing by `|F|!` gives the probability of that ordering.

The cost is exponential: `2^|F|` subsets. For a dataset with 50 features, that is 10^15 subsets. Unusable directly.

Approximations: KernelSHAP and TreeSHAP

KernelSHAP (Lundberg & Lee, 2017) approximates Shapley values by sampling a manageable number of subsets and fitting a weighted linear regression. It is model-agnostic — works for any `f`. It is slow: computing one local explanation requires hundreds of model evaluations.

TreeSHAP (Lundberg et al., 2020) exploits the tree structure of decision trees, random forests, and gradient-boosted trees. By walking the tree and keeping track of how each feature affects each path, TreeSHAP computes exact Shapley values in polynomial time: `O(T * L * D^2)` where `T` is number of trees, `L` is max leaves, `D` is max depth. For a typical LightGBM model (500 trees, depth 8), TreeSHAP computes SHAP values for an entire dataset in seconds.

Always prefer TreeSHAP when the model is tree-based. It is exact, not an approximation, and orders of magnitude faster than KernelSHAP.

SHAP Plots

Summary plot: one row per feature, sorted by mean absolute SHAP value. Each row is a scatter of points colored by feature value. A wide spread of SHAP values means the feature matters; color shows direction. This is the best single visualisation for model interpretation.

Dependence plot: SHAP value on the y-axis against feature value on the x-axis, one dot per sample. Reveals non-linear effects and interactions. Color by a second feature to see interactions visually.

Waterfall plot: for a single prediction, show how the features push the prediction from the baseline to the final value. The most intuitive local explanation — non-technical stakeholders understand it immediately.

Force plot: similar to waterfall but horizontal. Shows features pushing left (lower prediction) or right (higher prediction).

LIME: Local Interpretable Model-Agnostic Explanations

LIME (Ribeiro et al., 2016) takes a different approach. To explain a prediction for a single point `x`:

1. Perturb `x` by sampling `N` points near it (with Gaussian noise or by randomly replacing feature values).
2. Predict labels for all `N` perturbed points using the original (complex) model.
3. Fit a simple interpretable model (usually sparse linear regression) to the `N` predictions, weighted by proximity to `x`.
4. The coefficients of the simple model are the explanation.

LIME's pros: works for any model, any data type (text, images, tabular). Visualisations are intuitive. Cons: explanations are local only (no global importance summary), sensitive to the perturbation strategy, and less theoretically grounded than SHAP.

LIME vs SHAP in practice. For tabular models, SHAP with TreeSHAP is strictly better: exact, fast, principled. For image and text models where perturbations are the only option, both SHAP and LIME use sampling; SHAP has better theory but LIME's intuitive output is sometimes preferred for stakeholder communication.

Fairness: The Impossibility Theorem

Suppose you have a credit scoring model and two groups `A` and `B` (say, by ethnicity). You want the model to be "fair". What does fair mean?

- **Demographic parity**: the fraction predicted positive should be equal across groups. `P(y_hat = 1 | G = A) = P(y_hat = 1 | G = B)`.
- **Equalized odds**: true positive rate and false positive rate should be equal across groups. `P(y_hat = 1 | y = 1, G = A) = P(y_hat = 1 | y = 1, G = B)` and similarly for `y = 0`.
- **Calibration parity**: predicted probabilities should be equally reliable across groups. `P(y = 1 | p_hat = p, G = A) = P(y = 1 | p_hat = p, G = B) = p`.

These all sound reasonable. The problem: **when the base rate of the positive outcome differs across groups, you cannot satisfy all three simultaneously**. This is the impossibility theorem (Chouldechova, 2017; Kleinberg et al., 2016).

Sketch of the argument. Consider two groups with different base rates. Suppose the model is calibrated (calibration parity holds). Then within each group, among the people with predicted probability `p`, exactly fraction `p` are actually positive. Now count: the false positive rate is `FP / (FP + TN)`. Calibration forces this ratio to follow the base rate. If base rates differ, so do FPRs. So equalized odds fails. Symmetric arguments show that enforcing equalized odds breaks calibration.

The theorem is a mathematical fact: you must choose. If two groups have genuinely different base rates, you cannot have a model that is both calibrated and has equal error rates. You must pick which fairness criterion matters most for your application and accept that others will be violated.

Disparate impact ratio. A common regulatory measure:

```
DIR = P(y_hat = 1 | G = minority) / P(y_hat = 1 | G = majority)
```

US employment law's "four-fifths rule" says DIR must be at least 0.8. Singapore's Tripartite Guidelines on Fair Employment Practices apply similar principles. If DIR is below 0.8, the model is potentially subject to disparate impact claims and needs to be either justified (a business necessity) or mitigated.

Fairness as Engineering

Fairness is not a property you can assume — it must be measured, reported, and if necessary mitigated.

1. **Measure**: compute disparate impact, equalized odds, and calibration across protected groups before deployment.
2. **Report**: include fairness metrics in the model card (Lesson 3.8).
3. **Mitigate where possible**: reweight training data, apply threshold adjustments per group, or constrain the optimisation during training.
4. **Accept when impossible**: document which criteria you chose to prioritise and why.

Mitigation is not always appropriate. If the base rates are different because of genuine differences in risk (say, group A has a genuinely higher default rate for economic reasons), forcing demographic parity means denying loans to qualified members of group B. That may be a worse outcome than the original

disparity. Fairness is a policy choice, not a technical one — engineers measure and report, stakeholders decide.

Kailash Engine: ModelVisualizer with SHAP

```
import shap
from kailash_ml import ModelVisualizer

explainer = shap.TreeExplainer(trained_model)
shap_values = explainer.shap_values(X_test)

# Global: summary plot
shap.summary_plot(shap_values, X_test, feature_names=feature_names)

# Local: waterfall
shap.plots.waterfall(shap.Explanation(
    values=shap_values[0],
    base_values=explainer.expected_value,
    data=X_test[0],
    feature_names=feature_names,
))
```

Worked Example: SHAP and Fairness Audit on Credit Scoring

Following MLFP03 ex_6:

```
import numpy as np
import polars as pl
import shap
import lightgbm as lgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
from shared import MLFPDataLoader

loader = MLFPDataLoader()
credit = loader.load("mlfp02", "credit_scoring.parquet")

# Keep a protected attribute for the fairness audit
protected_col = "gender"  # one of the columns in the dataset

feature_cols = [c for c in credit.columns if c not in ("default",)]
X = credit.select(feature_cols).to_numpy()
y = credit.select("default").to_series().to_numpy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42,
)

model = lgb.LGBMClassifier(n_estimators=500, learning_rate=0.05, random_state=42)
model.fit(X_train, y_train)
```

Compute TreeSHAP values:

```
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# Verify the additivity property
predictions = model.predict(X_test, raw_score=True)
sums = explainer.expected_value + shap_values.sum(axis=1)
assert np.allclose(predictions, sums, atol=1e-4), "SHAP additivity violated"
print("SHAP additivity verified: phi sum + baseline == raw model output")
```

Global importance ranking:

```
mean_abs_shap = np.abs(shap_values).mean(axis=0)
order = np.argsort(-mean_abs_shap)
print("Top 10 features by mean |SHAP|:")
for idx in order[:10]:
    print(f"  {feature_cols[idx]:30s} {mean_abs_shap[idx]:.4f}")
```

Local explanation for one customer:

```
i = 0
print(f"\nCustomer {i}: predicted probability = {model.predict_proba(X_test[[i]])[0, 1]:.3f}")
print(f"Baseline (expected): {1 / (1 + np.exp(-explainer.expected_value)):.3f}")

contribs = [(feature_cols[j], X_test[i, j], shap_values[i, j])
            for j in range(len(feature_cols))]
contribs.sort(key=lambda t: -abs(t[2]))

print("Top 5 features for this prediction:")
for name, value, phi in contribs[:5]:
```

```
    direction = "↑" if phi > 0 else "↓"
    print(f"  {direction} {name:25s}  value={value:.2f}  phi={phi:+.3f}")
```

Fairness audit:

```
protected_idx = feature_cols.index(protected_col)
group_values = X_test[:, protected_idx].astype(int)

for g in np.unique(group_values):
    mask = group_values == g
    auc = roc_auc_score(y_test[mask], model.predict_proba(X_test[mask])[:, 1])
    positive_rate = model.predict(X_test[mask]).mean()
    base_rate = y_test[mask].mean()
    print(f"Group {g}: n={mask.sum()}, base rate={base_rate:.3f}, "
          f"predicted positive rate={positive_rate:.3f}, AUC={auc:.3f}")

# Disparate impact ratio
pos_rates = {g: model.predict(X_test[group_values == g]).mean() for g in np.unique(group_values)}
minority = min(pos_rates, key=pos_rates.get)
majority = max(pos_rates, key=pos_rates.get)
dir_ratio = pos_rates[minority] / pos_rates[majority]
print(f"Disparate impact ratio: {dir_ratio:.3f}  (threshold: 0.80)")
if dir_ratio < 0.8:
    print("  WARNING: model may violate disparate impact four-fifths rule")
```

Mean SHAP by group for a protected feature:

```
for g in np.unique(group_values):
    mask = group_values == g
    mean_shap_per_feature = shap_values[mask].mean(axis=0)
    top = np.argsort(-np.abs(mean_shap_per_feature))[:5]
    print(f"\nGroup {g}: top 5 features by mean SHAP")
    for j in top:
        print(f"  {feature_cols[j]:25s} mean phi={mean_shap_per_feature[j]:+.4f}")
```

A striking difference in feature contributions between groups is evidence that the model is using different "reasoning" for different populations — sometimes an early warning of disparate impact, sometimes an honest reflection of true differences. Investigation is required.

Try It Yourself

Exercise A (easy). Generate a SHAP summary plot for the credit model. Which feature has the highest mean absolute SHAP value? Is that feature intuitively important?

Exercise B (medium). Compare TreeSHAP and KernelSHAP for the same model on 20 test points. Are the SHAP values numerically close? How much longer does KernelSHAP take?

Exercise C (hard). Implement a simple LIME-style explanation: given a model, a point `x`, and the training data, sample 500 points by randomly perturbing `x`, predict with the original model, then fit a `Lasso(alpha=0.01)` weighted by distance. Return the non-zero coefficients as the explanation. Compare to SHAP for the same point.

Cross-References

- **Lesson 3.4** Gradient boosting — TreeSHAP runs on these models.
- **Lesson 3.5** Calibration — calibration parity is one fairness criterion from the impossibility theorem.
- **Forward link:** Lesson 3.8 requires SHAP-based explanations in the model card.

Deeper Dive: Why Shapley Values Sum to the Prediction

The efficiency axiom — Shapley values sum to the total contribution — has a concrete interpretation for ML:

```
Sum_j phi_j = f(x) - E[f(X)]
```

That is, the SHAP values for one observation sum to the difference between this observation's prediction and the average prediction over the dataset. The "base value" `E[f(X)]` is the model's prior; each SHAP value is a deviation from it.

This is why SHAP waterfall plots start at the base value and end at the prediction: each feature's contribution pushes the running total up or down, and all contributions together exactly bridge the gap. If SHAP values did not satisfy efficiency, the waterfall would not add up, and the story would be incoherent.

Deeper Dive: Computing SHAP for a Single Tree

For a single decision tree, the SHAP value of feature `j` for observation `x` can be computed by this conceptual algorithm:

1. Walk from the root to a leaf using `x`. Record the prediction at the leaf.
2. At each internal node that tested feature `j`, consider the "counterfactual": what if we did not know `x_j`?
3. Replace the decision at that node with a weighted average of the two branches, weighted by the training-set proportions going each way.
4. The difference between the original prediction and the counterfactual is (part of) the SHAP value of `j`.

TreeSHAP (Lundberg et al., 2020) generalises this idea to an exact algorithm that handles the combinatorics correctly. The details involve tracking "hot" and "cold" feature subsets through the tree in a single pass, giving `O(T L D^2)` complexity per observation.

Deeper Dive: Local Accuracy and Consistency

Beyond the original four axioms, SHAP satisfies two additional properties that are particularly important for ML interpretation:

- **Local accuracy**: the sum of SHAP values for a single prediction equals the difference between that prediction and the baseline. (This is the efficiency axiom restated.)
- **Consistency**: if a model changes so that a feature's marginal contribution increases (in every context), that feature's SHAP value should not decrease.

Consistency rules out certain naive attribution methods. For example, the "gain" metric in XGBoost's feature importance fails consistency: you can modify a model so that a feature contributes more to every split, but its gain can decrease because the structure of the tree changes. SHAP values never fail consistency. This is why SHAP is preferred over traditional tree importance metrics.

Deeper Dive: Fairness Mitigation Techniques

When a fairness audit reveals disparate impact, you have several mitigation options:

Pre-processing: modify the training data before training. Reweigh samples to equalise group representation, or use fair representation learning to project features into a space that is "independent" of the protected attribute.

In-processing: modify the training objective. Add a fairness penalty to the loss, or use adversarial training where a second network tries to predict the protected attribute from the main model's predictions — if it succeeds, the main model is penalised.

Post-processing: adjust model outputs after training. Apply different thresholds per group to equalise TPR or FPR. Simple, easy to audit, but requires group membership at inference time (which may not be legal or desirable).

Each has trade-offs. Post-processing is the easiest to deploy but the least principled. Pre-processing is the easiest to audit but may lose important information. In-processing is the most thorough but hardest to reason about. The impossibility theorem means no mitigation fully solves the problem — you choose which criterion to prioritise.

Reflection Questions

1. Why are the four Shapley axioms necessary? Drop one — what goes wrong?
2. TreeSHAP is exact for tree models but KernelSHAP is approximate. Why is the approximation often "good enough"?
3. Your credit model has disparate impact ratio 0.75 but equalized odds holds. Should you mitigate? What are the trade-offs?
4. State the impossibility theorem in your own words, in one sentence.
5. The consistency property rules out XGBoost's "gain" feature importance. What happens if you use gain instead of SHAP and then change the model slightly?
6. A stakeholder asks: "What's the single most important feature for this model?" Explain why that question is ambiguous and what three distinct answers you might give.

Lesson 3.7 — Workflow Orchestration, Model Registry, and Hyperparameter Search

Why This Matters

A Singapore insurance company had a great model: a LightGBM fraud detector with 0.88 AUC. They trained it on a Monday. They deployed it on a Wednesday. On Thursday, the data science team found a bug in the feature engineering code. On Friday, when they tried to retrain with the fix, nobody could remember which exact combination of preprocessing steps, hyperparameters, and data splits produced the deployed model. They spent three weeks trying to reproduce the original training run. They never fully succeeded.

This is the reproducibility crisis in ML. It is not a problem that better algorithms solve — it is a problem that **workflows** solve. A workflow captures the entire pipeline from raw data to deployed model as a reproducible graph. Every training run is an execution of that graph. Every deployed model is linked back to the graph version that produced it. Every model has an immutable record: the data version, the code version, the hyperparameters, the metrics.

Kailash's `WorkflowBuilder` is this capture mechanism. Combined with the `ModelRegistry` for versioning and `HyperparameterSearch` for tuning, it turns one-off notebooks into production pipelines you can re-run a year later and get the same answer.

This lesson covers:

- How to build ML workflows with `WorkflowBuilder`.
- Custom nodes: `@register_node`, `Node` subclasses, `PythonCodeNode`.
- Bayesian optimisation for hyperparameter search.
- Model registry lifecycle: stage → promote → retire.
- `ModelSignature` for input/output validation.

Core Concepts

The Workflow Abstraction

A Kailash workflow is a directed acyclic graph of nodes. Each node does one thing: load data, preprocess, train, evaluate, register. Edges carry data between nodes. The runtime executes the graph in dependency order, passing outputs of upstream nodes as inputs to downstream nodes.

```
from kailash.workflow.builder import WorkflowBuilder
from kailash.runtime.local import LocalRuntime

workflow = WorkflowBuilder()
workflow.add_node("DataLoaderNode", "load", {"path": "credit.parquet"})
workflow.add_node("PreprocessNode", "prep", {"scaler": "standard"})
workflow.add_node("TrainNode", "train", {"model": "lightgbm"})
workflow.add_node("EvalNode", "eval", {"metrics": ["auc", "ap"]})

workflow.add_connection("load", "data", "prep", "input_data")
workflow.add_connection("prep", "processed", "train", "train_data")
workflow.add_connection("train", "model", "eval", "model")
workflow.add_connection("prep", "processed", "eval", "test_data")

runtime = LocalRuntime()
results, run_id = runtime.execute(workflow.build())
```

The four-argument `add_connection` pattern is core to Kailash: `(source_node, source_output, target_node, target_input)`. It creates an edge that carries the named output of one node to the named input of another.

Custom Nodes

When a built-in node does not exist for your task, create one.

Option 1: inline PythonCodeNode. Fast for one-off logic:

```
workflow.add_node("PythonCodeNode", "custom_feature", {
    "code": """
import polars as pl
result = {'features': input_data.with_columns(
    (pl.col('income') / pl.col('debt_ratio')).alias('income_per_debt_unit')
)}
"""
})
```

Option 2: Subclass Node. Cleaner and testable:

```
from kailash.nodes.base import Node, NodeParameter
from kailash.nodes.base import register_node

@register_node()
class CustomFeatureNode(Node):
    def get_parameters(self):
        return {
            "input_data": NodeParameter(name="input_data", type=pl.DataFrame, required=True),
        }

    def run(self, **kwargs):
        df = kwargs["input_data"]
        out = df.with_columns(
            (pl.col("income") / pl.col("debt_ratio")).alias("income_per_debt_unit")
        )
        return {"features": out}
```

Once registered, `CustomFeatureNode` is available in any workflow by name: `workflow.add_node("CustomFeatureNode", "feat", {})`.

Logic Nodes: Branching and Conditional Execution

Production workflows often need to branch based on runtime conditions. “If AUC > 0.85, promote to staging; else flag for review.” Kailash provides conditional nodes for this:

```
workflow.add_node("ConditionalNode", "gate", {
    "condition": "auc > 0.85",
})
workflow.add_connection("eval", "metrics", "gate", "inputs")
workflow.add_connection("gate", "true_branch", "promote", "signal")
workflow.add_connection("gate", "false_branch", "flag", "signal")
```

When `gate` fires, it routes execution to either `promote` or `flag` depending on the condition. This is how you encode the “human-on-the-loop” gate: automated promotion for high-confidence runs, human review for borderline runs.

Hyperparameter Search

Hyperparameters are the knobs you set before training. Their values matter enormously and cannot be learned from the data directly. Search strategies:

- **Grid search**: try every combination in a predefined grid. Exhaustive but expensive. `O(prod(k_i))` where `k_i` is the number of values for parameter `i`.

- **Random search**: sample random combinations. Surprisingly effective; Bergstra and Bengio (2012) showed random beats grid when only a few hyperparameters matter.
- **Bayesian optimisation**: model the hyperparameter → score relationship with a probabilistic surrogate (Gaussian process, tree-based Parzen estimator) and choose the next point to try based on expected improvement.

Bayesian Optimisation in Depth

The idea: treat the validation score as a black-box function of the hyperparameters, `f(theta)`. Build a probabilistic model of `f` from the points tried so far. Use the model to pick the next `theta` that is likely to improve the score.

Gaussian process surrogate. A Gaussian process defines a prior over functions. Given observed points `(theta_1, f(theta_1)), ..., (theta_n, f(theta_n))`, the posterior gives you a mean and variance at every unobserved point. Points with high mean are "likely good"; points with high variance are "uncertain".

Acquisition function. The acquisition function decides where to sample next, balancing exploration (high variance) and exploitation (high mean). Common choices:

- **Expected Improvement (EI)**: `EI(theta) = E[max(0, f(theta) - f_best)]`. Samples points likely to exceed the current best.
- **Upper Confidence Bound (UCB)**: `UCB(theta) = mu(theta) + beta * sigma(theta)`. Explicit exploration-exploitation trade-off via `beta`.
- **Probability of Improvement (PI)**: simpler but less exploratory.

Loop. (1) Build a surrogate from points tried so far. (2) Maximise the acquisition function over the hyperparameter space to pick the next point. (3) Evaluate `f` at that point. (4) Repeat.

For high-dimensional hyperparameter spaces, Tree-structured Parzen Estimator (TPE, Bergstra et al. 2011) often outperforms Gaussian processes. TPE is what Optuna uses by default.

Practical budget. Bayesian optimisation needs fewer trials than random search to reach a given score. Typical budget: 30–100 trials for 3–5 hyperparameters. Beyond 10 hyperparameters, gains diminish and it is often faster to use informed defaults.

Kailash HyperparameterSearch

```
from kailash_ml import HyperparameterSearch, SearchSpace, ParamDistribution, SearchConfig

space = SearchSpace(
    learning_rate=ParamDistribution.log_uniform(1e-3, 1e-1),
    num_leaves=ParamDistribution.int_uniform(15, 127),
    min_child_samples=ParamDistribution.int_uniform(5, 100),
    reg_lambda=ParamDistribution.log_uniform(1e-3, 1e2),
)

config = SearchConfig(
    strategy="bayesian",
    n_trials=50,
    cv_folds=5,
    scoring="average_precision",
    random_state=42,
)

searcher = HyperparameterSearch(space=space, config=config)
best = searcher.fit(X_train, y_train)
print(f"Best AP: {best.score:.3f}")
print(f"Best params: {best.params}")
```

The engine wraps Optuna internally. Parameters are sampled from the declared distributions, evaluated with cross-validation, and the surrogate model is updated after each trial. After `n_trials`, it returns the best configuration.

Model Registry: Lifecycle Management

A `ModelRegistry` stores trained models along with metadata, versioning, and lifecycle state. The state machine:

```
  Experiment   →   Staging   →   Production   →   Archive
      |             |              |
  (register)    (promote)      (retire)
```

- **Experiment**: just trained. Not yet validated for any use.
- **Staging**: passed validation, ready for A/B testing or shadow deployment.
- **Production**: serving real traffic.
- **Archive**: retired but kept for compliance and rollback.

```
from kailash_ml import ModelRegistry, ModelSignature, MetricSpec

registry = ModelRegistry(path="./model_registry")

sig = ModelSignature(
    inputs={"X": "float64[:, :]"},
    outputs={"proba": "float64[:]"},
)

metrics = MetricSpec(values={
```

```
    "auc": 0.87,
    "ap": 0.61,
    "brier": 0.08,
})

version = registry.register(
    model=trained_model,
    name="credit_default_lgb",
    signature=sig,
    metrics=metrics,
    metadata={
        "training_data": "credit_scoring_v3.parquet",
        "feature_version": "v2.1",
        "trained_at": "2026-04-10T14:23:00",
    },
)
print(f"Registered as version {version}")
```

Promotion between stages:

```
registry.promote("credit_default_lgb", version=version, stage="staging")
# ... some validation passes ...
registry.promote("credit_default_lgb", version=version, stage="production")
```

Rollback:

```
previous = registry.get("credit_default_lgb", stage="production", version="v42")
registry.promote("credit_default_lgb", version="v42", stage="production")
```

ModelSignature: The Contract

A `ModelSignature` defines the input and output schema. At serve time, the registry validates incoming requests against the signature and rejects anything mismatched. This is your first line of defence against production bugs like “client sends strings where we expected floats” or “client sends 15 features when we trained on 14”.

```
sig = ModelSignature(
    inputs={
        "age": "int64",
        "income": "float64",
        "debt_ratio": "float64",
        "num_credit_lines": "int64",
    },
    outputs={"default_probability": "float64"},
    constraints={
        "age": {"min": 18, "max": 120},
        "income": {"min": 0, "max": 1e7},
    },
)
```

The registry will refuse to serve a request with `age = 10` — it violates the constraint. That protects you from a client bug silently sending garbage.

Worked Example: End-to-End Workflow

Following MLFP03 ex_7:

```
import asyncio
import polars as pl
from kailash.workflow.builder import WorkflowBuilder
from kailash.runtime.local import LocalRuntime
from kailash_ml import (
    PreprocessingPipeline, TrainingPipeline, HyperparameterSearch,
    ModelRegistry, ModelSignature, SearchSpace, ParamDistribution, SearchConfig,
)
import lightgbm as lgb
from shared import MLFPDataLoader

loader = MLFPDataLoader()

# Build the workflow
workflow = WorkflowBuilder()

workflow.add_node("PythonCodeNode", "load", {
    "code": """
from shared import MLFPDataLoader
loader = MLFPDataLoader()
df = loader.load('mlfp02', 'credit_scoring.parquet')
result = {'data': df}
"""
})

workflow.add_node("PythonCodeNode", "split", {
    "code": """
```

```
from sklearn.model_selection import train_test_split
import numpy as np
X = data.drop('default').to_numpy()
y = data.select('default').to_series().to_numpy()
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
result = {'X_train': X_tr, 'y_train': y_tr, 'X_test': X_te, 'y_test': y_te}
"""
})

workflow.add_node("PythonCodeNode", "train", {
    "code": """
import lightgbm as lgb
model = lgb.LGBMClassifier(n_estimators=300, learning_rate=0.05, num_leaves=31, random_state=42)
model.fit(X_train, y_train)
result = {'model': model}
"""
})

workflow.add_node("PythonCodeNode", "evaluate", {
    "code": """
from sklearn.metrics import roc_auc_score, average_precision_score, brier_score_loss
proba = model.predict_proba(X_test)[:, 1]
result = {
    'auc': float(roc_auc_score(y_test, proba)),
    'ap': float(average_precision_score(y_test, proba)),
    'brier': float(brier_score_loss(y_test, proba)),
}
"""
})

workflow.add_connection("load", "data", "split", "data")
workflow.add_connection("split", "X_train", "train", "X_train")
workflow.add_connection("split", "y_train", "train", "y_train")
workflow.add_connection("split", "X_test", "evaluate", "X_test")
workflow.add_connection("split", "y_test", "evaluate", "y_test")
workflow.add_connection("train", "model", "evaluate", "model")

runtime = LocalRuntime()
results, run_id = runtime.execute(workflow.build())
print(f"Run ID: {run_id}")
print(f"Metrics: {results['evaluate']}")
```

Every run gets a unique `run_id`. Every intermediate artefact (data, model, metrics) is stored against that ID. Re-running the same workflow gives you a new ID but the same lineage — you can diff runs to find where behaviour diverged.

Hyperparameter Search with the Workflow

```
space = SearchSpace(
    learning_rate=ParamDistribution.log_uniform(1e-3, 1e-1),
    num_leaves=ParamDistribution.int_uniform(15, 127),
    min_child_samples=ParamDistribution.int_uniform(5, 100),
    reg_lambda=ParamDistribution.log_uniform(1e-3, 1e2),
)
config = SearchConfig(strategy="bayesian", n_trials=40, cv_folds=5,
                      scoring="average_precision", random_state=42)

searcher = HyperparameterSearch(space=space, config=config, base_model_cls=lgb.LGBMClassifier)
best = searcher.fit(X_train, y_train)
print(f"Best params: {best.params}")
print(f"Best AP: {best.score:.3f}")

final_model = lgb.LGBMClassifier(**best.params, random_state=42)
final_model.fit(X_train, y_train)
```

Registry and Promotion

```
registry = ModelRegistry(path="./registry")

sig = ModelSignature(
    inputs={f"feature_{i}": "float64" for i in range(X_train.shape[1])},
    outputs={"default_probability": "float64"},
)

version = registry.register(
    model=final_model,
    name="mlfp_credit_default",
    signature=sig,
    metrics={"auc": 0.87, "ap": 0.61},
    metadata={"hyperparameters": best.params, "run_id": run_id},
)

registry.promote("mlfp_credit_default", version=version, stage="staging")
```

Try It Yourself

Exercise A (easy). Build a workflow that loads the HDB data, trains a `LinearRegression`, and reports R-squared. Add a conditional node that promotes to staging only if R-squared > 0.7.

Exercise B (medium). Compare random search and Bayesian optimisation on LightGBM for the credit dataset. Both get 30 trials. Report the best AP each achieves. Plot the cumulative-best-so-far curve for both. Does Bayesian win?

Exercise C (hard). Subclass `Node` to create a `SMOTENode` that performs SMOTE resampling. Register it with `@register_node`. Use it in a workflow between `split` and `train`.

Cross-References

- **Module 1, Lesson 1.8** Python classes and inheritance — used for subclassing `Node`.
- **Lesson 3.4** Gradient boosting — the model we are tuning here.
- **Lesson 3.5** Metrics — what we optimise during search.
- **Forward link:** Lesson 3.8 uses the model registered here for production deployment.

Deeper Dive: Bayesian Optimisation with Expected Improvement

The Expected Improvement acquisition function is the mathematical workhorse of Bayesian optimisation. At a point `theta` where the surrogate has posterior mean `mu(theta)` and standard deviation `sigma(theta)`, and the best observed score so far is `f_best`, EI is:

```
EI(theta) = E[max(0, f(theta) - f_best)]
         = (mu(theta) - f_best) * Phi(Z) + sigma(theta) * phi(Z)
```

where `Z = (mu(theta) - f_best) / sigma(theta)`, `Phi` is the standard normal CDF, and `phi` is the standard normal PDF.

Two intuitions:

- If `mu` is much larger than `f_best` (likely better than current best), EI is large — exploit this region.
- If `sigma` is large (uncertain), EI is also large — explore even if mean is not great.

EI naturally balances the two. Points with high mean dominate when the model is confident; points with high variance dominate when the model is unsure. As the search progresses and the surrogate becomes more accurate, EI concentrates on high-mean regions — the search converges.

For the log of a probability target (e.g., AUC), the EI formula is applied to the raw score. For minimisation (e.g., log loss), negate the score.

Deeper Dive: Tree-Structured Parzen Estimator (TPE)

For high-dimensional hyperparameter spaces with conditional structure (e.g., "if optimizer is Adam, then tune beta_1; if optimizer is SGD, then tune momentum"), Gaussian processes struggle. TPE (Bergstra et al., 2011) is the alternative Optuna uses by default.

TPE splits observed points into "good" (top `gamma` fraction, say top 15%) and "bad" (the rest). It fits two density estimators:

- `l(theta)` = density over good parameters
- `g(theta)` = density over bad parameters

The acquisition function is `l(theta) / g(theta)` — the ratio of good-density to bad-density at `theta`. Points where good parameters concentrate and bad parameters are rare are preferred.

TPE scales better than GPs and handles conditional and categorical parameters naturally. For most ML hyperparameter search, TPE is the right default.

Deeper Dive: The Workflow as a Reproducibility Artefact

A Kailash workflow is more than an execution plan — it is a reproducibility artefact. When you call `runtime.execute(workflow.build())`, the runtime captures:

- The workflow DAG structure.
- The parameters of every node.
- The order of execution and timing.
- The inputs and outputs of every node (or references to them in the data store).
- The run ID, a unique identifier for this execution.

Six months later, you can take the run ID and rebuild the exact workflow that produced a specific model, with the exact parameters. You cannot do this with a notebook — notebooks have global state, cell ordering that can be rerun out of order, and hidden dependencies on what was last in memory. Workflows do not.

This is why production ML systems move from notebooks to workflows as they mature. A notebook is fine for exploration. A workflow is required for production.

Reflection Questions

1. Why is the `run_id` from `runtime.execute()` important for reproducibility?
2. When would you use `PythonCodeNode` instead of a custom `Node` subclass? When the reverse?

3. Bayesian optimisation has an exploration-exploitation trade-off. What acquisition function parameter controls it, and what does tuning it do?
4. A `ModelSignature` constraint rejects `age = 10`. A legitimate client in Malaysia sends such a request for a special-case product. How do you handle this without breaking the contract?
5. Derive Expected Improvement from the definition `E[max(0, f(theta) - f_best)]` assuming `f(theta)` is Gaussian with known mean and variance.
6. Why does TPE handle conditional parameters better than Gaussian processes?

Lesson 3.8 — Production Pipeline: DataFlow, Drift, and Deployment

Why This Matters

A Singapore telco deployed a customer churn model in January 2020. It was performing brilliantly. Then COVID-19 hit. Roaming stopped. Data usage spiked as workers moved home. The feature distributions shifted dramatically. The model's predictions continued to be confident, but they were now confidently wrong. By March, churn predictions had decoupled entirely from actual churn. No alarm fired because the model never "errored" — it just quietly degraded. The telco lost an estimated S$8 million in retention budget spent on customers who did not actually churn before they detected the problem.

This is **model drift**. A model trained on yesterday's distribution can become useless when the world changes. Catching drift is not optional — it is the single most important operational task for a deployed ML model. You need to monitor:

- **Input drift**: are the feature distributions still what you trained on?
- **Prediction drift**: are the model's outputs still in the expected range?
- **Performance drift**: when ground truth arrives, does the model still score well?

This lesson teaches you how to build the full production pipeline: train, persist results to DataFlow, calibrate, register, monitor for drift, generate model cards, and package everything as a reproducible artefact. By the end, you will have a model that is not just accurate — it is auditable, monitored, and safe to trust.

Core Concepts

The Full Production Pipeline

A production ML pipeline has eight stages, four of which only exist once the model is live:

1. **Train** the model (Lessons 3.1–3.5).
2. **Calibrate** the output probabilities (Lesson 3.5).
3. **Register** the model with metadata (Lesson 3.7).
4. **Promote** to staging, then production (Lesson 3.7).
5. **Serve** predictions via an API.
6. **Log** every prediction to a database (DataFlow).
7. **Monitor** inputs, predictions, and — when available — outcomes (DriftMonitor).
8. **Retire** when replaced or when drift exceeds tolerance.

Stages 5–8 are what "production" means. Without them, you have a notebook, not a product.

DataFlow: Persistence for ML Results

Kailash's `DataFlow` is a zero-config database layer. You declare a schema with `@db.model`, and DataFlow generates CRUD operations automatically. It handles connection pooling, schema migrations, and async I/O.

```
from kailash.db import ConnectionManager, db
from kailash.db.models import field

@db.model
class ModelEvaluation:
    id: int = field(primary_key=True)
    model_name: str = field(index=True)
    model_version: str
    dataset_version: str
    auc: float
    average_precision: float
    brier_score: float
    calibration_error: float
    trained_at: str
    metadata_json: str

@db.model
class PredictionLog:
    id: int = field(primary_key=True)
    request_id: str = field(index=True)
    model_name: str = field(index=True)
    model_version: str
    features_json: str
    prediction: float
    predicted_at: str
    outcome: float = field(nullable=True)  # filled in later when ground truth arrives
```

Using these models:

```
async def persist_evaluation(conn, eval_data):
    return await db.express.create(conn, ModelEvaluation, eval_data)

async def get_history(conn, model_name):
    return await db.express.list(conn, ModelEvaluation,
                                 filter={"model_name": model_name},
                                 order_by="-trained_at")

async def update_outcome(conn, request_id, outcome):
    return await db.express.update(conn, PredictionLog,
                                   filter={"request_id": request_id},
                                   values={"outcome": outcome})
```

`db.express` is the built-in convenience API: `create`, `list`, `get`, `update`, `delete`. For complex queries, drop to raw SQL via the connection manager, but 80% of ML bookkeeping is satisfied by `db.express` alone.

DriftMonitor: Watching the World Change

Drift detection uses statistical tests to compare the distribution of incoming features to a reference (usually the training distribution). Two tests dominate: **PSI** and **KS**.

PSI (Population Stability Index)

PSI measures how much a distribution has shifted. For a feature with observed proportions `P_actual` in each bucket and baseline proportions `P_baseline`:

```
PSI = Sum_buckets (P_actual - P_baseline) * ln(P_actual / P_baseline)
```

This is the symmetric version of KL divergence: it is the KL divergence `D(P_actual || P_baseline) + D(P_baseline || P_actual)`, both with the same log. The higher the PSI, the more the distribution has shifted.

Typical thresholds:

- `PSI < 0.1`: no significant change.
- `0.1 <= PSI < 0.25`: moderate change; investigate but do not necessarily act.
- `PSI >= 0.25`: major change; the model is potentially unreliable and should be retrained.

Computing PSI in practice. Bucket the feature into 10 deciles using the training data quantiles as bucket boundaries. For each bucket, compute `P_actual` from the live data and `P_baseline` from the training data. Apply the formula. Clamp each `P_actual` to a minimum of, say, 0.0001 to avoid log-of-zero when a bucket is empty.

```
import numpy as np

def psi(actual, baseline, bins=10):
    quantiles = np.quantile(baseline, np.linspace(0, 1, bins + 1))
    quantiles[0] -= 1e-9
    quantiles[-1] += 1e-9
    actual_counts, _ = np.histogram(actual, bins=quantiles)
    baseline_counts, _ = np.histogram(baseline, bins=quantiles)
    actual_prop = np.clip(actual_counts / max(actual_counts.sum(), 1), 1e-4, None)
    baseline_prop = np.clip(baseline_counts / max(baseline_counts.sum(), 1), 1e-4, None)
    return np.sum((actual_prop - baseline_prop) * np.log(actual_prop / baseline_prop))
```

KS Test (Kolmogorov-Smirnov)

KS measures the maximum distance between two empirical cumulative distribution functions:

```
D = max_x |F_actual(x) - F_baseline(x)|
```

where `F_actual` and `F_baseline` are the ECDFs of the two samples. The test statistic `D` is the largest vertical gap between the two step functions.

KS has a known null distribution: if both samples come from the same distribution, `sqrt(n) * D` converges to the Kolmogorov distribution. The scipy function `scipy.stats.ks_2samp` returns both `D` and a p-value.

- `p-value < 0.01`: strong evidence of drift.
- `p-value >= 0.01`: no significant drift.

KS is more sensitive than PSI for detecting small-scale distribution changes, but it does not tell you the direction or magnitude of the shift. Use both: PSI for a continuous "how much" score, KS for a statistical test.

Performance Drift

Input drift is a leading indicator — it may not directly harm predictions if the model is robust. Performance drift is the lagging indicator: when ground truth arrives, how does the model actually score?

Monitor rolling AUC, rolling AP, and rolling log loss over a sliding window (e.g., 30 days). Alert when any metric drops by more than 5% relative to the training benchmark.

The challenge is that ground truth often arrives late. For credit default, you need 30–180 days before you know if a loan defaulted. So performance drift is measured with a lag, while input drift is measured immediately. Combine both for full coverage.

DriftSpec

Kailash's `DriftMonitor` takes a `DriftSpec` declaring which features to monitor, which tests to run, and which thresholds trigger alerts:

```
from kailash_ml import DriftMonitor, DriftSpec

spec = DriftSpec(
    features={
        "age": {"test": "psi", "threshold": 0.25},
        "income": {"test": "ks", "threshold": 0.05},
        "debt_ratio": {"test": "psi", "threshold": 0.25},
    },
    baseline_data=X_train,
    monitoring_frequency="daily",
    alert_channels=["logs", "metrics"],
)

monitor = DriftMonitor(spec=spec)
alerts = monitor.check(new_batch)
```

If any feature breaches its threshold, the monitor emits an alert that downstream systems can route to Slack, PagerDuty, or a retraining queue.

Conformal Prediction

Standard classifiers give you a probability. Conformal prediction gives you a **set of plausible outputs with a coverage guarantee**. Specifically: "with probability at least 90%, the true label is in this prediction set" — and this guarantee holds regardless of the underlying model, regardless of the data distribution, as long as calibration and test points are exchangeable.

The recipe (split conformal prediction).

1. Train the model on a training set.
2. On a held-out calibration set of size `n`, compute a non-conformity score for each point — e.g., `s_i = 1 - p_hat(y_i | x_i)` for classification.
3. Compute the `ceil((n + 1) * (1 - alpha)) / n` quantile of the `s_i`. Call it `q`.
4. For a new point `x`, the prediction set is `{y : 1 - p_hat(y | x) <= q}`.

This set has coverage `1 - alpha` in expectation, regardless of the model. `alpha = 0.1` gives 90% coverage.

For regression, use `s_i = |y_i - y_hat(x_i)|`, and the prediction interval is `[y_hat(x) - q, y_hat(x) + q]`.

Conformal prediction is the only method that gives distribution-free coverage guarantees. It is the right tool when downstream decisions need principled uncertainty quantification — think medical diagnosis, legal risk assessments, safety-critical systems.

Model Cards

A model card (Mitchell et al., 2019) is a document that accompanies every deployed model. It answers the questions a regulator, auditor, or new team member would ask:

1. **Model details**: name, version, type, training date, owner.
2. **Intended use**: what problems this model is designed to solve, what it is NOT designed for.
3. **Factors**: relevant demographic or contextual factors that affect performance.
4. **Metrics**: headline performance numbers, confidence intervals, fairness metrics.
5. **Evaluation data**: what dataset was used for evaluation, how it was collected.
6. **Training data**: provenance, size, known biases.
7. **Quantitative analyses**: disaggregated performance across groups.
8. **Ethical considerations**: known failure modes, fairness concerns, mitigation strategies.
9. **Caveats and recommendations**: what users should know before trusting the model.

Write the model card alongside the model. If you cannot write one, you do not understand the model well enough to deploy it.

MLOps: Clean Architecture for ML

The end state of production ML is what industry calls "MLOps" — continuous integration, continuous delivery, and continuous monitoring applied to ML. Key practices:

- **Version everything**: code, data, features, models, metrics.
- **Automate training and evaluation**: every merge to main triggers a retraining run.
- **Shadow deployment**: new models serve predictions alongside production, but predictions are only logged, not acted on. After a week of shadow data, compare shadow to production.
- **Canary releases**: gradually shift traffic from old to new model (10% → 50% → 100%).
- **Rollback on metric drop**: if live metrics degrade, automatically revert to the previous version.
- **Model retirement**: models not serving traffic for 90 days are archived; metadata is kept for audit.

These practices turn one-off ML projects into a sustainable engineering function.

Worked Example: End-to-End Production Pipeline

Following MLFP03 ex_8:

```python
import asyncio
from datetime import datetime
import json
import numpy as np
import polars as pl
import lightgbm as lgb
from sklearn.model_selection import train_test_split
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import (
    roc_auc_score, average_precision_score, brier_score_loss, log_loss,
)
from kailash.db import ConnectionManager
from kailash.db.models import db, field
from kailash_ml import ModelRegistry, ModelSignature, DriftMonitor, DriftSpec
from shared import MLFPDataLoader

loader = MLFPDataLoader()
credit = loader.load("mlfp02", "credit_scoring.parquet")

feature_cols = [c for c in credit.columns if c != "default"]
X = credit.select(feature_cols).to_numpy()
y = credit.select("default").to_series().to_numpy()

X_train, X_holdout, y_train, y_holdout = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42,
)
X_cal, X_test, y_cal, y_test = train_test_split(
    X_holdout, y_holdout, test_size=0.5, stratify=y_holdout, random_state=42,
)
```

Step 1: Train and calibrate.

```python
base = lgb.LGBMClassifier(
    n_estimators=500, learning_rate=0.05, num_leaves=31,
    subsample=0.8, colsample_bytree=0.8, random_state=42,
)
calibrated = CalibratedClassifierCV(base, method="isotonic", cv=5)
calibrated.fit(X_train, y_train)

p_test = calibrated.predict_proba(X_test)[:, 1]
auc = roc_auc_score(y_test, p_test)
ap = average_precision_score(y_test, p_test)
brier = brier_score_loss(y_test, p_test)
ll = log_loss(y_test, p_test)
print(f"Test AUC={auc:.3f}, AP={ap:.3f}, Brier={brier:.4f}, LogLoss={ll:.3f}")
```

Step 2: Conformal prediction for uncertainty.

```python
alpha = 0.10
p_cal = calibrated.predict_proba(X_cal)
y_cal_int = y_cal.astype(int)
nonconformity = 1.0 - p_cal[np.arange(len(y_cal_int)), y_cal_int]

q_level = np.ceil((len(nonconformity) + 1) * (1 - alpha)) / len(nonconformity)
q_level = min(q_level, 1.0)
q_hat = np.quantile(nonconformity, q_level)
print(f"Conformal quantile q_hat = {q_hat:.3f} for coverage {1 - alpha:.0%}")

p_test_full = calibrated.predict_proba(X_test)
prediction_sets = [np.where(1.0 - p_test_full[i] <= q_hat)[0] for i in range(len(X_test))]

coverage = np.mean([y_test[i] in prediction_sets[i] for i in range(len(X_test))])
avg_set_size = np.mean([len(s) for s in prediction_sets])
print(f"Empirical coverage: {coverage:.3f} (target {1 - alpha:.0%})")
print(f"Average set size: {avg_set_size:.2f}")
```

The empirical coverage should be close to 90%. If it is not, something is wrong with exchangeability (e.g., you accidentally used training data as calibration).

Step 3: Persist to DataFlow.

```python
@db.model
class ModelRunLog:
    id: int = field(primary_key=True)
    model_name: str = field(index=True)
    run_id: str
    auc: float
    ap: float
    brier: float
    log_loss: float
    coverage: float
    trained_at: str

async def persist():
    conn = ConnectionManager.get_default()
    run_data = {
```

```
        "model_name": "mlfp_credit_default_v1",
        "run_id": datetime.utcnow().strftime("%Y%m%dT%H%M%S"),
        "auc": auc,
        "ap": ap,
        "brier": brier,
        "log_loss": ll,
        "coverage": coverage,
        "trained_at": datetime.utcnow().isoformat(),
    }
    await db.express.create(conn, ModelRunLog, run_data)
    all_runs = await db.express.list(conn, ModelRunLog,
                                     filter={"model_name": "mlfp_credit_default_v1"})
    return all_runs

all_runs = asyncio.run(persist())
print(f"Total runs in registry: {len(all_runs)}")
```

Step 4: Register in ModelRegistry.

```
registry = ModelRegistry(path="./registry")

sig = ModelSignature(
    inputs={f: "float64" for f in feature_cols},
    outputs={"default_probability": "float64"},
)

version = registry.register(
    model=calibrated,
    name="mlfp_credit_default_v1",
    signature=sig,
    metrics={"auc": auc, "ap": ap, "brier": brier},
    metadata={
        "training_size": len(X_train),
        "conformal_coverage": coverage,
        "conformal_q_hat": float(q_hat),
        "calibration_method": "isotonic",
    },
)

registry.promote("mlfp_credit_default_v1", version=version, stage="staging")
print(f"Registered version: {version}")
```

Step 5: Set up drift monitoring.

```
def psi(actual, baseline, bins=10):
    quantiles = np.quantile(baseline, np.linspace(0, 1, bins + 1))
    quantiles[0] -= 1e-9
    quantiles[-1] += 1e-9
    actual_counts, _ = np.histogram(actual, bins=quantiles)
    baseline_counts, _ = np.histogram(baseline, bins=quantiles)
    a = np.clip(actual_counts / max(actual_counts.sum(), 1), 1e-4, None)
    b = np.clip(baseline_counts / max(baseline_counts.sum(), 1), 1e-4, None)
    return float(np.sum((a - b) * np.log(a / b)))

# Simulate drift: shift income distribution in the test batch
X_drifted = X_test.copy()
income_idx = feature_cols.index("income") if "income" in feature_cols else 1
X_drifted[:, income_idx] = X_drifted[:, income_idx] * 1.5 + 10000

print("\n=== Drift Report ===")
for i, name in enumerate(feature_cols[:8]):  # limit to first 8 for display
    psi_no_drift = psi(X_test[:, i], X_train[:, i])
    psi_drift = psi(X_drifted[:, i], X_train[:, i])
    marker = " ** DRIFT **" if psi_drift > 0.25 else ""
    print(f"  {name:25s}  no-drift PSI={psi_no_drift:.3f}  drifted PSI={psi_drift:.3f}{marker}")
```

When you inject a 50% shift and offset in `income`, its PSI should jump from near zero to well above 0.25, triggering an alert. The other features should stay below threshold.

Step 6: Model card.

```
model_card = {
    "model_details": {
        "name": "mlfp_credit_default_v1",
        "version": version,
        "type": "Calibrated LightGBM",
        "owner": "MLFP Credit Team",
        "trained_at": datetime.utcnow().isoformat(),
    },
    "intended_use": {
        "primary": "Predict probability of credit default on personal loan applications",
        "out_of_scope": [
            "Corporate credit",
            "Secured loans (mortgage, auto)",
```

```
            "Decisions without human review",
        ],
    },
    "factors": {
        "demographic": ["age", "gender"],
        "geographic": ["Singapore only"],
        "temporal": ["Training data covers 2022-2024"],
    },
    "metrics": {
        "auc": round(auc, 3),
        "average_precision": round(ap, 3),
        "brier": round(brier, 4),
        "log_loss": round(ll, 3),
        "conformal_coverage_90": round(coverage, 3),
    },
    "evaluation_data": {
        "size": len(X_test),
        "base_rate": float(y_test.mean()),
        "source": "Anonymized Singapore credit bureau data",
    },
    "training_data": {
        "size": len(X_train),
        "base_rate": float(y_train.mean()),
        "known_biases": [
            "Under-representation of self-employed (8% vs Singapore census 11%)",
            "Limited coverage of age > 65",
        ],
    },
    "ethical_considerations": {
        "fairness_audited": True,
        "disparate_impact_ratio": 0.87,
        "impossibility_tradeoff": "Prioritised calibration parity; equalized odds may vary",
    },
    "caveats": [
        "Do not use for automated rejection without human review",
        "Retrain quarterly or when PSI > 0.25 on any feature",
        "Monitor conformal coverage monthly",
    ],
}

with open("model_card.json", "w") as f:
    json.dump(model_card, f, indent=2)
print("Model card written to model_card.json")
```

Step 7: Promote to production.

```
# Only after stakeholder sign-off
registry.promote("mlfp_credit_default_v1", version=version, stage="production")
print("Model promoted to production")
```

Try It Yourself

Exercise A (easy). Compute PSI for each feature in the credit dataset, comparing the first half of the data (as baseline) to the second half (as actual). Any features with PSI > 0.1?

Exercise B (medium). Implement a KS-based drift detector. Use `scipy.stats.ks_2samp` and return the p-values. Compare the set of features it flags to the set PSI flags. Are they the same?

Exercise C (hard). Build a simulated production stream: every day, draw 200 new samples, add small Gaussian noise to one feature, pass them through the calibrated model, compute rolling AUC (with delayed labels arriving 30 days later), and compute daily PSI. Plot all three metrics over a 90-day window. At what day does drift become detectable?

Cross-References

- **Lesson 3.5** Calibration — the calibrated model is what we serve in production.
- **Lesson 3.6** Fairness — the fairness audit feeds the model card.
- **Lesson 3.7** Registry — the pipeline ends with a promoted model.
- **Module 2, Lesson 2.2** Hypothesis testing — KS is a hypothesis test.

Deeper Dive: Conformal Coverage Guarantee

The coverage guarantee for split conformal prediction is:

```
P(y_new in C(x_new)) >= 1 - alpha
```

where `C(x_new)` is the conformal prediction set. The guarantee is marginal (averaged over the randomness of calibration and test draws) and requires only that the calibration and test points are **exchangeable** — equivalent to "i.i.d. from the same distribution" for most practical purposes.

Why it works. Let `s_i = s(x_i, y_i)` be the non-conformity score on the calibration set, and `s_new = s(x_new, y_new)` the score on the test point. Under exchangeability, the test point is statistically indistinguishable from a calibration point. The rank of `s_new` among `(s_1, ..., s_n, s_new)` is uniform over `{1, 2, ..., n+1}`. So:

```
P(s_new <= q_hat) = rank_cutoff / (n + 1)
```

Choosing `q_hat` as the `ceil((n+1)(1-alpha))/n` quantile of the calibration scores gives the desired coverage.

The guarantee is tight: you cannot do better than this without stronger assumptions. You can do worse if you violate exchangeability (e.g., distribution shift between calibration and test, or adaptive thresholds).

Deeper Dive: Drift Detection Cadence

How often should you check for drift? The answer depends on:

- **Data arrival rate**. A daily batch job can check daily. A real-time stream needs a different architecture.
- **Drift velocity**. How fast can the world change in your domain? For consumer behaviour, days. For scientific instruments, months. For financial markets, minutes.
- **Cost of missed drift**. High-stakes models (credit, fraud) should check more often.
- **Cost of false alarms**. If every drift alarm triggers a retraining, false alarms are expensive.

Typical cadences:

- **Daily**: check PSI on features, check prediction distribution, alert if thresholds breached.
- **Weekly**: recompute rolling performance metrics (when ground truth is available), refresh drift baselines.
- **Monthly**: retrain baseline model as a benchmark; compare current production to freshly trained.
- **Quarterly**: full re-training and re-calibration, regardless of drift signals. Prevents slow drift accumulation.

Combine leading and lagging signals. Input drift (PSI, KS) is leading — it tells you something changed. Performance drift (AUC drop) is lagging — it tells you the model got worse. You need both.

Deeper Dive: Why Three Splits?

The split in MLFP03 ex_8 uses train, calibration, and test. Why three?

- **Train**: fit the model parameters.
- **Calibration**: fit the probability calibrator (Platt / isotonic) and/or the conformal quantile. Must be separate from train because the model is biased on training data — its predictions on train data are over-confident.
- **Test**: evaluate the final (calibrated) model honestly. Must be separate from calibration because the calibrator has seen calibration labels.

With only two splits (train, test), you either:

1. Calibrate on train — biased, over-confident predictions.
2. Calibrate on test — calibration metrics are no longer honest; test set contaminated by calibration.

Three splits is the minimum for an honest production pipeline with calibrated probabilities.

Deeper Dive: What a Complete Model Card Looks Like

The Mitchell et al. (2019) template specifies nine sections. Here are the questions each section answers:

1. **Model Details**: Name? Version? Type? Who owns it? When was it trained?
2. **Intended Use**: What is the model designed to do? Who are the intended users? What are the out-of-scope uses?
3. **Factors**: What demographic, phenotypic, environmental, or instrumentation factors affect performance?
4. **Metrics**: What metrics are reported? What thresholds were used? How were confidence intervals computed?
5. **Evaluation Data**: What datasets were used? How was evaluation data collected? Any preprocessing?
6. **Training Data**: Same questions, for training. Any known biases?
7. **Quantitative Analyses**: Performance disaggregated across the factors from section 3 (e.g., per gender, per age bracket).
8. **Ethical Considerations**: Privacy, fairness, safety, societal impact. What could go wrong?
9. **Caveats and Recommendations**: What should users know? When should they not trust the model?

Writing a model card is a forcing function. If you cannot answer these questions, the model is not ready to deploy.

Reflection Questions

1. Why do we use three different data splits (train / calibration / test)? What would go wrong with only two?
2. Conformal prediction has a distribution-free coverage guarantee. What assumption is required for the guarantee to hold?
3. PSI thresholds of 0.1 and 0.25 are conventions, not proofs. How would you choose thresholds for a new application?
4. Your model card says “do not use for automated rejection”. A product manager removes this clause to ship faster. What do you do?
5. Derive the conformal coverage guarantee sketch from the exchangeability assumption and the rank argument.
6. Your drift monitor alerts three times a week. All three alerts turn out to be false alarms. What do you change?

Module 3 Closing

You have now walked the entire supervised ML pipeline: from the raw data that enters your laptop to a calibrated, monitored, documented model serving traffic with a drift detector watching its back. Each lesson added a piece:

- Lesson 3.1 taught you to engineer and select features without leaking the future into the past.
- Lesson 3.2 gave you the bias-variance decomposition, regularisation, and the CV strategies that match your data structure.
- Lesson 3.3 added five classical models to your vocabulary: SVM, KNN, Naive Bayes, Decision Trees, and Random Forests.
- Lesson 3.4 went deep on gradient boosting — the dominant tabular algorithm — with XGBoost's second-order derivation, LightGBM's GOSS, and CatBoost's ordered boosting.
- Lesson 3.5 confronted the realities of imbalance, evaluation metrics, and calibration.
- Lesson 3.6 gave you two tools for explanation (SHAP and LIME) and the impossibility theorem that governs fairness.
- Lesson 3.7 turned your training code into a reproducible workflow with hyperparameter search and a model registry.
- Lesson 3.8 closed the loop with production persistence, drift monitoring, conformal prediction, and model cards.

The next module (MLFP04) takes you into the unsupervised world: clustering, dimensionality reduction, anomaly detection, and the natural language processing techniques that sit between classical ML and deep learning. Module 5 brings deep learning and transformers; Module 6 is LLMs and agents.

But the skeleton you built here — engineer, regularise, choose, tune, evaluate, interpret, register, monitor — is the skeleton of every ML system you will ever build. Know it by heart. Come back to this textbook when you lose your way.

Appendix Z: Worked Numerical Examples

These worked examples let you verify your understanding with pencil and paper before running any code.

Z.1: Computing Bias and Variance from Bootstrap Samples

Suppose the true function is `f(x) = 3x + 2`. At `x = 5`, the true value is `y = 17`. You have a noisy dataset and fit 10 linear regression models on different bootstrap samples. Their predictions at `x = 5` are:

```
y_hat = [15.2, 16.8, 17.5, 16.3, 18.1, 14.9, 17.9, 16.6, 15.8, 17.2]
```

Compute:

- **Mean prediction**: `(15.2 + 16.8 + ... + 17.2) / 10 = 166.3 / 10 = 16.63`.
- **Bias**: `16.63 - 17 = -0.37`. **Bias squared**: `0.137`.
- **Variance**: `(1/10) * Sum (y_hat_i - 16.63)^2`:
  - Deviations squared: `(15.2 - 16.63)^2 = 2.04`, `(16.8 - 16.63)^2 = 0.029`, `(17.5 - 16.63)^2 = 0.757`, `(16.3 - 16.63)^2 = 0.109`, `(18.1 - 16.63)^2 = 2.161`, `(14.9 - 16.63)^2 = 2.993`, `(17.9 - 16.63)^2 = 1.613`, `(16.6 - 16.63)^2 = 0.001`, `(15.8 - 16.63)^2 = 0.689`, `(17.2 - 16.63)^2 = 0.325`.
  - Sum = 10.717. Variance = 10.717 / 10 = 1.072.

At `x = 5`, this model has small bias (|–0.37|) and moderate variance (1.07). Most of the expected squared error comes from variance, not bias. The right remedy is more regularisation or more training data, not a more flexible model.

Z.2: XGBoost Split Gain with Explicit Numbers

A node contains 6 samples. Before the split, their gradients and hessians are:

```
sample:  1    2     3     4     5     6
g:      -0.9 -0.7  -0.3  0.2   0.6   0.8
h:       0.2  0.2   0.2  0.2   0.2   0.2
```

Parent totals: `G = -0.9 - 0.7 - 0.3 + 0.2 + 0.6 + 0.8 = -0.3`. `H = 1.2`.

With `lambda = 1` and `gamma = 0.1`, consider splitting at feature threshold so that samples 1, 2, 3 go left and 4, 5, 6 go right.

- Left: `G_L = -0.9 - 0.7 - 0.3 = -1.9`. `H_L = 0.6`. Term: `(-1.9)^2 / (0.6 + 1) = 3.61 / 1.6 = 2.256`.
- Right: `G_R = 0.2 + 0.6 + 0.8 = 1.6`. `H_R = 0.6`. Term: `(1.6)^2 / 1.6 = 1.6`.
- Parent term: `(-0.3)^2 / (1.2 + 1) = 0.09 / 2.2 = 0.041`.

Gain:

```
Gain = (1/2) * [2.256 + 1.6 - 0.041] - 0.1 = (1/2) * 3.815 - 0.1 = 1.9075 - 0.1 = 1.8075
```

Positive and substantial — the split is worth making. The optimal leaf weights:

- `w_L* = -G_L / (H_L + lambda) = 1.9 / 1.6 = 1.1875`.
- `w_R* = -G_R / (H_R + lambda) = -1.6 / 1.6 = -1.0`.

Left leaf adds 1.19 to its samples' scores; right leaf subtracts 1.0.

Z.3: PSI Computation with Actual Numbers

Suppose in training you had income distributed across 10 deciles with 100 samples per decile (all equal). In production, income shifts: the first decile now has 150 samples and the tenth decile has only 50, with other deciles unchanged at 100 each. Total new samples: `150 + 100*8 + 50 = 1000`.

Baseline proportions: `P_baseline = [0.1] * 10`.

Actual proportions: `P_actual = [0.15, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.05]`.

PSI:

```
Bucket 1:  (0.15 - 0.10) * log(0.15 / 0.10) = 0.05 * 0.4055 = 0.02027
Buckets 2-9: (0.10 - 0.10) * log(1.0) = 0 each = 0 total
Bucket 10: (0.05 - 0.10) * log(0.05 / 0.10) = -0.05 * -0.6931 = 0.03466

PSI = 0.02027 + 0 + 0.03466 = 0.0549
```

PSI of 0.055 is below the 0.1 threshold — no significant drift. Now imagine a more dramatic shift: `P_actual = [0.3, 0.1, 0.1, 0.1, 0.1, 0.1, 0.05, 0.05, 0.05, 0.05]`.

```
Bucket 1:  (0.30 - 0.10) * log(3) = 0.20 * 1.0986 = 0.2197
Bucket 7:  (0.05 - 0.10) * log(0.5) = -0.05 * -0.6931 = 0.0347
Buckets 8-10: same as bucket 7 = 0.0347 each, total 0.1041
Bucket 7-10 combined: 0.1388

Sum: 0.2197 + 0.1388 = 0.3585
```

PSI of 0.36 — well above the 0.25 threshold. Major drift. Alert fires, retraining triggered.

Z.4: Conformal Quantile Calculation

You have 100 calibration samples with non-conformity scores:

```
s = [0.01, 0.03, 0.05, ..., 0.99] (sorted)
```

For 90% coverage (`alpha = 0.1`), the quantile level is:

```
q_level = ceil((100 + 1) * (1 - 0.1)) / 100 = ceil(90.9) / 100 = 91 / 100 = 0.91
```

So you take the 91st smallest score. If the sorted scores at position 91 is 0.83, then `q_hat = 0.83`. For a new point `x_new` with predicted probabilities over two classes `[0.2, 0.8]`, the non-conformity score for class 0 is `1 - 0.2 = 0.8` and for class 1 is `1 - 0.8 = 0.2`. Both are below `q_hat = 0.83`, so both classes are in the prediction set: `C(x_new) = {0, 1}`. The prediction set is "both possible" — no confident prediction.

For a more decisive point with probabilities `[0.05, 0.95]`: - Class 0 score: `1 - 0.05 = 0.95 > 0.83` — excluded. - Class 1 score: `1 - 0.95 = 0.05 < 0.83` — included.

Prediction set is `{1}` alone — singleton, a confident prediction.

This is the power of conformal prediction: the set size automatically reflects the model's certainty. Uncertain predictions get large sets; confident predictions get singletons.

Appendix A: End-to-End Singapore Case Study

To cement the ideas from the eight lessons, walk through this end-to-end case study. It is not an exercise — it is a reading exercise. The goal is to see how every decision connects.

The Problem

DBS Bank (hypothetically) wants to build a personal loan default prediction model for the Singapore market. The business context:

- **Loan product**: unsecured personal loans, S$5,000 to S$100,000, 12 to 60 month terms.
- **Base rate**: 8% of loans default within 24 months.
- **Regulatory context**: MAS Notice 635 requires explainable credit decisions. PDPA gives consumers the right to ask for an explanation.
- **Business cost**: average loss on default is S$22,000. Average profit on a good loan is S$1,800. The cost ratio is 12:1.

Lesson 3.1: Features

The raw data has 60 columns across three tables: applications, credit bureau pulls, and transaction history. The feature engineer:

1. Joins the three tables on `customer_id` and `application_date`.
2. Computes bureau-based features: number of open lines, total outstanding balance, credit utilisation, number of late payments in the past 24 months.
3. Computes transaction-based features: average monthly income (from salary deposits), income variability (coefficient of variation), cash-out-to-income ratio.
4. Computes temporal features: customer tenure, days since last product opened, months of transaction history.
5. Avoids leakage: every feature is computed with data strictly before `application_date`. The rolling features use `closed="left"` in polars group_by_dynamic.

Result: 87 features, registered in FeatureStore as `credit_default_v3`.

Lesson 3.2: Bias-Variance and Regularisation

Initial baseline: L2-regularised logistic regression (`Ridge`-style). Train MSE is low, validation MSE is slightly higher — the model has moderate variance. A nested 5×3 CV gives AP = 0.41 (vs base rate 0.08).

Lesson 3.3: Model Zoo Comparison

The team tries five models using the same 5-fold stratified CV:

Model	AP	Log Loss	Training Time
Logistic (L2)	0.41	0.21	0.3s
SVM RBF	0.48	—	45s
KNN (k=15)	0.32	—	2s
Naive Bayes	0.28	0.35	0.1s
Random Forest	0.54	0.19	12s

Random Forest wins. SVM is competitive but too slow for production retraining. Logistic is the interpretable fallback.

Lesson 3.4: Gradient Boosting

XGBoost, LightGBM, and CatBoost all improve further:

Model	AP	Log Loss	Time
XGBoost	0.59	0.17	25s
LightGBM	0.60	0.17	8s
CatBoost	0.61	0.17	60s

LightGBM picked for production: best speed/accuracy trade-off. CatBoost slightly better on AP but 7x slower.

Lesson 3.5: Imbalance and Calibration

With base rate 8%, the team:

1. Trains LightGBM with `scale_pos_weight = (1 - 0.08) / 0.08 = 11.5` (cost-sensitive).
2. Applies isotonic calibration via `CalibratedClassifierCV(method="isotonic", cv=5)`.
3. Computes the cost-optimal threshold: `p* = c_FP / (c_FP + c_FN) = 1800 / (1800 + 22000) = 0.0756`.
4. Evaluates on held-out test: AP 0.61, Brier 0.048, reliability curve close to diagonal.

Lesson 3.6: Interpretability and Fairness

SHAP summary plot shows the top features:

1. `credit_utilisation` — higher utilisation, higher default.
2. `months_since_last_late_payment` — longer since late, lower default.
3. `income_coefficient_of_variation` — higher variability, higher default.
4. `total_outstanding_balance` — higher balance, higher default.

Fairness audit across gender:

- Male disparate impact ratio: 1.00 (baseline group).
- Female disparate impact ratio: 0.94 — within the 0.8 threshold but lower.
- Equalized odds: male TPR 0.61, female TPR 0.58; male FPR 0.11, female FPR 0.09.

The team notes the small TPR gap and documents it in the model card. No mitigation applied; the gap is small enough to be consistent with genuine risk differences.

Lesson 3.7: Workflow and Hyperparameter Search

The full pipeline is encoded as a Kailash workflow with nodes for load → split → preprocess → train → calibrate → evaluate → register. Bayesian hyperparameter search with TPE runs 80 trials over `learning_rate`, `num_leaves`, `min_child_samples`, `reg_lambda`, improving AP from 0.60 to 0.63.

The tuned model is registered as `dbs_credit_default` version 1.0.0 in ModelRegistry, with signature, metrics, and metadata.

Lesson 3.8: Production

Before promoting to staging:

1. Conformal prediction calibrated at 90% coverage gives a `q_hat` of 0.21 on the calibration set.
2. DriftSpec configures PSI monitoring on all 87 features, with thresholds of 0.15 (warning) and 0.25 (alert).
3. Model card written, 9 sections, reviewed by legal and compliance.
4. DataFlow `ModelEvaluation` and `PredictionLog` tables deployed; every scoring request writes a row.

The model goes to staging for shadow deployment. For two weeks, it scores every application in parallel with the existing model; predictions are logged but not used. After two weeks, the team compares shadow metrics to production metrics. Shadow wins. Canary rollout begins: 10% of traffic, then 50% after 48 hours, then 100% after another 48 hours.

Two months later, PSI on `total_outstanding_balance` jumps to 0.28. Investigation reveals that a new credit bureau started contributing data with a slightly different definition of “outstanding balance”. The model is retrained on the updated data, re-registered as version 1.1.0, and promoted after passing the same

shadow-canary pipeline.

This is the full MLOps lifecycle. Every lesson in this module was a step in this story.

Appendix B: Formulas Reference

Bias-Variance:

```
E[(y - y_hat)^2] = Bias^2(y_hat) + Var(y_hat) + sigma^2
```

Regularisation:

```
L_ridge = Sum (y_i - x_i^T beta)^2 + lambda * Sum beta_j^2
L_lasso = Sum (y_i - x_i^T beta)^2 + lambda * Sum |beta_j|
L_enet  = Sum (y_i - x_i^T beta)^2 + lambda * (alpha * Sum |beta_j| + (1-alpha) * Sum beta_j^2)
```

Ridge closed form:

```
beta_ridge = (X^T X + lambda I)^(-1) X^T y
```

SVM dual:

```
maximise   Sum alpha_i - (1/2) Sum_{i,j} alpha_i alpha_j y_i y_j K(x_i, x_j)
subject to Sum alpha_i y_i = 0,  0 <= alpha_i <= C
```

Gini and Entropy:

```
G = 1 - Sum_k p_k^2
H = -Sum_k p_k log p_k
```

Information Gain:

```
IG = H(parent) - Sum_j (|child_j|/|parent|) H(child_j)
```

Random Forest OOB:

```
P(not in bootstrap) = (1 - 1/n)^n -> 1/e ≈ 0.368
```

XGBoost split gain:

```
Gain = (1/2) [ G_L^2/(H_L + lambda) + G_R^2/(H_R + lambda) - (G_L + G_R)^2/(H_L + H_R + lambda) ] - gamma
```

Optimal leaf weight:

```
w_j^* = -G_j / (H_j + lambda)
```

Focal loss:

```
FL(p_t) = -alpha_t * (1 - p_t)^gamma * log(p_t)
```

Brier score:

```
BS = (1/N) Sum (p_i - y_i)^2
```

Log loss:

```
LL = -(1/N) Sum [y_i log(p_i) + (1 - y_i) log(1 - p_i)]
```

Classification metrics:

```
Precision = TP / (TP + FP)
Recall    = TP / (TP + FN)
F1        = 2 * (Precision * Recall) / (Precision + Recall)
TPR       = TP / (TP + FN)
FPR       = FP / (FP + TN)
```

Shapley value:

```
phi_i = Sum_{S subset F\{i}} [|S|!(|F|-|S|-1)!/|F|!] * [f(S ∪ {i}) - f(S)]
```

Efficiency:

```
Sum_i phi_i = f(all) - f(none)
```

Disparate impact ratio:

```
DIR = P(y_hat=1 | G=minority) / P(y_hat=1 | G=majority)
```

Four-fifths rule: `DIR >= 0.8`.

PSI:

```
PSI = Sum_buckets (P_actual - P_baseline) * log(P_actual / P_baseline)
```

Thresholds: `< 0.1` no change; `0.1 – 0.25` moderate; `>= 0.25` major.

KS statistic:

```
D = max_x |F_actual(x) - F_baseline(x)|
```

Cost-optimal threshold:

```
p* = c_FP / (c_FP + c_FN)
```

Expected Improvement (Bayesian optimisation):

```
EI(theta) = (mu - f_best) Phi(Z) + sigma phi(Z),   Z = (mu - f_best) / sigma
```

Conformal quantile:

```
q_hat = quantile_{ceil((n+1)(1-alpha))/n}(s_1, ..., s_n)
```

Appendix C: Kailash Engine Quick Reference

Engine	Purpose	Lesson
`DataExplorer`	Profile datasets, detect types, spot issues	3.1
`FeatureEngineer`	Generate derived features	3.1
`FeatureStore`	Version and serve features	3.1
`PreprocessingPipeline`	Scale, encode, impute	3.2
`TrainingPipeline`	Standard train + evaluate loop	3.3-5
`AutoMLEngine`	Multi-family automated search	3.4
`EnsembleEngine`	Stacking and blending	3.5
`ModelVisualizer`	SHAP plots, calibration curves	3.6
`HyperparameterSearch`	Bayesian / random / grid search	3.7
`ModelRegistry`	Versioning and lifecycle	3.7
`WorkflowBuilder`	Pipeline orchestration	3.7
`DataFlow`	Database persistence with schemas	3.8
`DriftMonitor`	PSI / KS drift detection	3.8
`ExperimentTracker`	Log runs, params, metrics	3.1-8

Appendix D: Further Reading

Foundational papers:

- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD*.
- Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS*.
- Prokhorenkova, L., et al. (2018). CatBoost: Unbiased Boosting with Categorical Features. *NeurIPS*.
- Lundberg, S., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS*.
- Lundberg, S., et al. (2020). From Local Explanations to Global Understanding with Explainable AI for Trees. *Nature Machine Intelligence*.
- Lin, T.-Y., et al. (2017). Focal Loss for Dense Object Detection. *ICCV*.
- Platt, J. (1999). Probabilistic Outputs for Support Vector Machines. *Advances in Large Margin Classifiers*.
- Mitchell, M., et al. (2019). Model Cards for Model Reporting. *FAT**.
- Chouldechova, A. (2017). Fair Prediction with Disparate Impact. *Big Data*, 5(2).
- Kleinberg, J., Mullainathan, S., & Raghavan, M. (2016). Inherent Trade-Offs in the Fair Determination of Risk Scores.

Textbooks for deeper study:

- Hastie, T., Tibshirani, R., & Friedman, J. *The Elements of Statistical Learning*.
- Murphy, K. *Probabilistic Machine Learning: An Introduction* and *Advanced Topics*.
- Barber, D. *Bayesian Reasoning and Machine Learning*.

Practical resources:

- Google's Machine Learning Crash Course — free, short, well-paced.
- fast.ai's Practical Deep Learning — focused on deep learning but the ML engineering advice is universal.
- Kaggle competitions and kernels — read winning solutions to see real decision-making.

End of MLFP Module 3 Textbook.